

Excel Smart Parser

Полный листинг исходного кода

Состав документа		
excel_smart_parser.py		1949 строк · 76.3 КБ · стр. 3
TEST/test_all_features.py		1622 строк · 74.1 КБ · стр. 25
excel_viewer/index.html		696 строк · 25.4 КБ · стр. 43

Всего строк кода	4267
Общий объём исходников	175.8 КБ
Страниц в документе	50
Кегль листинга	6.5 pt / интерлиньяж 7.8 pt
Дата сборки	08.09.2026 10:34

Контрольные суммы SHA-256

cda20c0282c4738b71d381e5e2f30bde103f5ac51cbd458ee698847bd3682719	excel_smart_parser.py
5e84dd243633efe51631a68ad327ba60fafe1c3a7958f3c25f4f35aa3231b1e9	TEST/test_all_features.py
b0bf12859e9a316f013199c412943268be02ebcc1cd976c7d438664df36e94c6	excel_viewer/index.html

Содержание

Ядро парсера — `excel_smart_parser.py`

3

Python · 1949 строк · 22 стр. листинга

Тесты — `TEST/test_all_features.py`

25

Python · 1622 строк · 18 стр. листинга

Веб-просмотрщик — `excel_viewer/index.html`

43

HTML · 696 строк · 8 стр. листинга

Ядро парсера — excel_smart_parser.py

excel_smart_parser.py · Python · 1949 строк · 76.3 КБ · sha256:cda20c0282c4738b

```

1  """
2  Excel Universal Parser v19
3  =====
4  Исправления относительно v18.1:
5
6  [FIX] Эвристика теперь запускается ВСЕГДА (не только при пустом all_tables).
7  Ранее если на листе были native/named tables, heuristic не запускался,
8  и остальные строки терялись. Теперь поведение единообразно: все 5
9  источников запускаются всегда, перекрытия предотвращает used_rows.
10
11 [FIX] _extract_heuristic / _extract_vertical / _extract_headerless:
12 двойное чтение ячеек для _detect_dtype. Ранее adapter.cell()
13 вызывался повторно в list comprehension для dtype, хотя значения
14 уже прочитаны при формировании rows_out. Добавлен col_values кэш
15 во всех трёх методах (ранее был только в _parse_range).
16
17 [FIX] CLI --include-hidden не работал: skip_hidden по умолчанию False,
18 а --include-hidden ставил skip_hidden=not True=False – то же самое.
19 Заменён на --skip-hidden (skip_hidden=True при указании флага).
20
21 [FIX] XlrdAdapter.cell(): import xlrd выполнялся внутри метода при каждом
22 вызове ячейки. Убран повторный import – модуль сохраняется в
23 self._xlrd при инициализации адаптера (один import в __init__).
24
25 [FIX] PyxlsbAdapter: named_ranges() открывал файл повторно через
26 pyxlsb.open_workbook(), хотя _ensure_cache() уже это делала.
27 Рефакторинг: named_ranges извлекаются в _ensure_cache() за один
28 проход и сохраняются в _named_ranges_cache.
29
30 [FIX] _serialize: не обрабатывал datetime.time и datetime.timedelta.
31 Добавлена сериализация: time -> .isoformat(), timedelta -> str().
32
33 [FIX] _detect_dtype: boolean значения (True/False) не распознавались –
34 _is_numeric возвращает False для bool, они попадали в "text".
35 Добавлен тип "boolean" при >50% bool-значений.
36
37 [FIX] CSV: пробел убран из KNOWN_DELIMITERS и _FALLBACK_DELIMITERS.
38 Пробел как разделитель ложно срабатывает на любом тексте с пробелами.
39
40 [FIX] _load_xlsx: при ошибке scan файл открывался до 3 раз (scan +
41 fallback + основной). Упрощено: при ошибке scan сразу переходим
42 к основному открытию без промежуточного fallback.
43
44 [FIX] ExcelParser.__init__: добавлена валидация параметров (threshold,
45 min_data_cells, max_empty_streak).
46
47 [FIX] Удалён deprecated _check_formulas_lazy – внутренняя функция,
48 не часть публичного API.
49
50 [FIX] Docstring changelog (194 строки) вынесен – модульный docstring
51 содержит только описание и версию.
52
53 [FIX] Дублирование кода сборки rows_out: вынесен общий метод
54 _build_rows() для _extract_heuristic, _extract_vertical,
55 _extract_headerless.
56 """
57
58
59 from __future__ import annotations
60
61 import abc
62 import argparse
63 import codecs
64 import csv
65 import datetime
66 import json
67 import math
68 import os
69 import sys
70 import traceback
71 import warnings
72 from typing import Any, Generator, Optional
73
74 # -- Обязательная зависимость -----
75 try:
76     import openpyxl
77     from openpyxl.utils import get_column_letter, column_index_from_string
78     from openpyxl.utils import range_boundaries as _range_boundaries
79 except ImportError:
80     sys.exit("pip install openpyxl")
81
82 # -- Опциональные зависимости -----
83 try:
84     from tqdm import tqdm as _tqdm
85     HAS_TQDM = True
86 except ImportError:
87     HAS_TQDM = False

```

```

88
89
90     try:
91         import xlrd
92         HAS_XLRD = True
93     except ImportError:
94         HAS_XLRD = False
95
96     try:
97         import pyxlsb
98         HAS_PYXLSB = True
99     except ImportError:
100         HAS_PYXLSB = False
101
102     try:
103         import chardet as _chardet
104         HAS_CHARDET = True
105     except ImportError:
106         try:
107             import charset_normalizer as _chardet # type: ignore
108             HAS_CHARDET = True
109         except ImportError:
110             HAS_CHARDET = False
111
112 # =====
113 # Типы
114 # =====
115
116 CellValue = Any
117 Row = dict[str, CellValue]
118
119 TABLE_SOURCE_NATIVE = "native_table"
120 TABLE_SOURCE_NAMED = "named_range"
121 TABLE_SOURCE_HEURISTIC = "heuristic"
122 TABLE_SOURCE_VERTICAL = "vertical"
123 TABLE_SOURCE_HEADERLESS = "headerless"
124
125
126 # =====
127 # Абстрактный адаптер листа
128 # =====
129
130 class SheetAdapter(abc.ABC):
131     """Абстрактный адаптер листа. Подклассы обязаны реализовать cell()."""
132     name: str
133     max_row: int
134     max_col: int
135
136     @abc.abstractmethod
137     def cell(self, row: int, col: int) -> CellValue:
138         """Возвращает значение ячейки (1-based row, col). None для пустых."""
139
140     def iter_rows_lazy(self, cols: list[int]) -> Generator[tuple[int, list[CellValue], None, None]:
141         for r in range(1, self.max_row + 1):
142             yield r, [self.cell(r, c) for c in cols]
143
144     def hidden_rows(self) -> set[int]:
145         return set()
146
147     def hidden_cols(self) -> set[int]:
148         return set()
149
150     def native_tables(self) -> list[dict]:
151         return []
152
153     def named_ranges(self) -> list[dict]:
154         return []
155
156
157 # -- openpyxl (.xlsx / .xlsm / .xltx / .xltm) -----
158
159 class OpenpyxlAdapter(SheetAdapter):
160     def __init__(self, ws, name: str, wb=None):
161         self._ws = ws
162         self._wb = wb # ссылка на workbook для закрытия
163         self.name = name
164         self.max_row = ws.max_row or 0
165         self.max_col = ws.max_column or 0
166         self._merged: dict[tuple[int, int], CellValue] = {}
167         for rng in ws.merged_cells.ranges:
168             master = ws.cell(rng.min_row, rng.min_col).value
169             for r in range(rng.min_row, rng.max_row + 1):
170                 for c in range(rng.min_col, rng.max_col + 1):
171                     self._merged[(r, c)] = master
172
173     def cell(self, row: int, col: int) -> CellValue:
174         if row < 1 or col < 1 or row > self.max_row or col > self.max_col:
175             return None
176         if (row, col) in self._merged:
177             return self._merged[(row, col)]
178         return self._ws.cell(row, col).value

```

```

179
180 def hidden_rows(self) -> set[int]:
181     return {r for r, d in self.ws.row_dimensions.items() if d.hidden}
182
183 def hidden_cols(self) -> set[int]:
184     result: set[int] = set()
185     for letter, d in self.ws.column_dimensions.items():
186         if d.hidden:
187             try:
188                 result.add(column_index_from_string(letter))
189             except Exception as e:
190                 warnings.warn(
191                     f"Лист '{self.name}': не удалось распознать скрытую колонку "
192                     f"'{letter}': {e}",
193                     UserWarning, stacklevel=2,
194                 )
195     return result
196
197 def close(self) -> None:
198     """Закрывает workbook, если адаптер хранит ссылку на него."""
199     if self._wb is not None:
200         try:
201             self._wb.close()
202         except Exception:
203             pass
204         self._wb = None
205
206 def native_tables(self) -> list[dict]:
207     tables = []
208     for tname, tobj in self.ws.tables.items():
209         try:
210             ref = tobj.ref if hasattr(tobj, "ref") else str(tobj)
211             if ":" not in ref:
212                 continue
213             min_col, min_row, max_col, max_row = _range_boundaries(ref)
214             tables.append({"name": tname, "ref": ref,
215                           "min_row": min_row, "max_row": max_row,
216                           "min_col": min_col, "max_col": max_col})
217         except Exception as e:
218             warnings.warn(
219                 f"Лист '{self.name}': нативная таблица '{tname}' "
220                 f"имеет некорректный диапазон: {e}",
221                 UserWarning, stacklevel=2,
222             )
223     return tables
224
225
226
227 # -- xlrd (.xls) -----
228
229 class XlrdAdapter(SheetAdapter):
230     def __init__(self, sheet, book, name: str):
231         import xlrd as _xlrd_module
232         self._xlrd = _xlrd_module
233         self._sheet = sheet
234         self._book = book
235         self.name = name
236         self.max_row = sheet.nrows
237         self.max_col = sheet.ncols
238
239     def cell(self, row: int, col: int) -> CellValue:
240         # FIX v19: используем self._xlrd вместо top-level xlrd
241         # (top-level xlrd может быть не определён если модуль не установлен)
242         if row < 1 or col < 1 or row > self.max_row or col > self.max_col:
243             return None
244         c = self._sheet.cell(row - 1, col - 1)
245         if c.ctype == self._xlrd.XL_CELL_EMPTY:
246             return None
247         if c.ctype == self._xlrd.XL_CELL_DATE:
248             try:
249                 return self._xlrd.xldate_as_datetime(c.value, self._book.datemode)
250             except Exception as e:
251                 warnings.warn(
252                     f"Лист '{self.name}': не удалось преобразовать дату "
253                     f"('{xldate={c.value}}': {e}). Возвращается raw float.",
254                     UserWarning, stacklevel=2,
255                 )
256             return c.value
257         return c.value
258
259     def named_ranges(self) -> list[dict]:
260         result = []
261         try:
262             for name_obj in self._book.name_obj_list:
263                 name = name_obj.name
264                 if not name_obj.result:
265                     continue
266                 try:
267                     for area in name_obj.result.coords:
268                         shtxlo, shtxhi, row0, row1, col0, col1 = area[:6]
269                         sheet_idx = shtxlo

```

```

270         if self._book.sheet_names()[sheet_idx] != self.name:
271             continue
272         # FIX: xlrd coords возвращает exclusive row1/col1,
273         # но _parse_range ожидает inclusive границы
274         result.append({
275             "name": name,
276             "sheet": self.name,
277             "min_row": row0 + 1,
278             "max_row": row1,      # row1 exclusive в xlrd = inclusive для 1-based
279             "min_col": col0 + 1,  # col1 exclusive в xlrd = inclusive для 1-based
280             "max_col": col1,
281         })
282     except Exception as e:
283         warnings.warn(
284             f"Лист '{self.name}': именованный диапазон '{name}' "
285             f"не удалось распарсить: {e}",
286             UserWarning, stacklevel=2,
287         )
288     except Exception as e:
289         warnings.warn(
290             f"Лист '{self.name}': ошибка при чтении name_obj_list: {e}",
291             UserWarning, stacklevel=2,
292         )
293     return result
294
295 # -- pyxlsb (.xlsb) -----
296
297 class PyxlsbAdapter(SheetAdapter):
298     """
299     Адаптер для .xlsb файлов. Данные хранятся в _row_cache.
300     Создаётся через _load_xlsb(), который открывает файл один раз
301     и читает все листы за один проход.
302     """
303
304     def __init__(
305         self,
306         sheet_name: str,
307         row_cache: dict[int, list],
308         max_row: int,
309         max_col: int,
310         named_ranges_cache: list[dict],
311     ):
312         self.name = sheet_name
313         self._row_cache = row_cache
314         self.max_row = max_row
315         self.max_col = max_col
316         self._named_ranges_cache = named_ranges_cache
317
318     def cell(self, row: int, col: int) -> CellValue:
319         if row < 1 or col < 1 or row > self.max_row or col > self.max_col:
320             return None
321         row_data = self._row_cache.get(row)
322         if row_data is None:
323             return None
324         idx = col - 1
325         if idx >= len(row_data):
326             return None
327         return row_data[idx]
328
329     def iter_rows_lazy(self, cols: list[int]) -> Generator[tuple[int, list[CellValue]], None, None]:
330         col_to_pos = {c: i for i, c in enumerate(cols)}
331         for r in range(1, self.max_row + 1):
332             row_data = self._row_cache.get(r, [None] * self.max_col)
333             vals = [None] * len(cols)
334             for c, pos in col_to_pos.items():
335                 idx = c - 1
336                 if 0 <= idx < len(row_data):
337                     vals[pos] = row_data[idx]
338             yield r, vals
339
340     def named_ranges(self) -> list[dict]:
341         # FIX v19: возвращаем кэш вместо повторного открытия файла
342         return self._named_ranges_cache
343
344 # -- CSV -----
345
346 def _coerce_csv_value(v: str) -> CellValue:
347     """Приводит строковое значение CSV к числовому типу, если возможно."""
348     if not v:
349         return None
350     # Boolean
351     vl = v.strip().lower()
352     if vl in ("true", "false"):
353         return vl == "true"
354     # FIX: NaN/Infinity - невалидны в JSON, возвращаем как строку
355     if vl in ("nan", "inf", "-inf", "+inf", "infinity", "-infinity", "+infinity"):
356         return v
357     # Числа: убираем пробелы
358     cleaned = v.strip().replace(" ", "")

```

```

361     if not cleaned:
362         return v
363     # Целое число
364     try:
365         return int(cleaned)
366     except ValueError:
367         pass
368     # FIX: обработка тысячных разделителей.
369     # "1,234,567" или "1 234 567" — запятые/пробелы как тысячные, точка — десятичная
370     # "1.234.567" или "1.234,56" — точки как тысячные, запятая — десятичная
371     # Стратегия: определяем роль запятой и точки по позиции последнего вхождения.
372     has_comma = "," in cleaned
373     has_dot = "." in cleaned
374     if has_comma and has_dot:
375         # Оба символа: последний из них — десятичный разделитель
376         last_comma = cleaned.rfind(",")
377         last_dot = cleaned.rfind(".")
378         if last_comma > last_dot:
379             # "1.234,56" — точка тысячная, запятая десятичная
380             try:
381                 return float(cleaned.replace(".", "").replace(",", ".", 1))
382             except ValueError:
383                 pass
384         else:
385             # "1,234.56" — запятая тысячная, точка десятичная
386             try:
387                 return float(cleaned.replace(",", ""))
388             except ValueError:
389                 pass
390     elif has_comma and not has_dot:
391         # Только запятые: если больше одной — тысячные ("1,234,567")
392         comma_count = cleaned.count(",")
393         if comma_count > 1:
394             try:
395                 return int(cleaned.replace(",", ""))
396             except ValueError:
397                 pass
398             try:
399                 return float(cleaned.replace(",", ""))
400             except ValueError:
401                 pass
402         else:
403             # Одна запятая — десятичный разделитель ("3,14")
404             try:
405                 return float(cleaned.replace(",", ".", 1))
406             except ValueError:
407                 pass
408     elif has_dot and not has_comma:
409         # Только точки: если больше одной — тысячные ("1.234.567")
410         dot_count = cleaned.count(".")
411         if dot_count > 1:
412             try:
413                 return int(cleaned.replace(".", ""))
414             except ValueError:
415                 pass
416             try:
417                 return float(cleaned.replace(".", ""))
418             except ValueError:
419                 pass
420         else:
421             # Одна точка — стандартный float
422             try:
423                 return float(cleaned)
424             except ValueError:
425                 pass
426     else:
427         # Нет ни запятых, ни точек — уже пробовали int выше.
428         # FIX: пробуем float для научной нотации (1e5, -1E10, 2e-3)
429         try:
430             f = float(cleaned)
431             if math.isfinite(f):
432                 return f
433             except ValueError:
434                 pass
435     return v
436
437
438 class CsvAdapter(SheetAdapter):
439     """
440     Адаптер CSV. Весь файл загружается в _row_cache одним проходом.
441     Потребление памяти O(n) по числу строк.
442     """
443
444     def __init__(self, filepath: str, encoding: str, delimiter: str, name: str):
445         self.name = name
446         self._filepath = filepath
447         self._encoding = encoding
448         self._delimiter = delimiter
449         self._row_cache: dict[int, list] = {}
450         self._max_row, self._max_col = self._load_cache()
451

```

```

452 def _load_cache(self) -> tuple[int, int]:
453     max_r = max_c = 0
454     with open(self.filepath, newline="", encoding=self.encoding) as f:
455         for i, row in enumerate(csv.reader(f, delimiter=self.delimiter)):
456             r = i + 1
457             max_r = r
458             if len(row) > max_c:
459                 max_c = len(row)
460             self._row_cache[r] = row
461     return max_r, max_c
462
463 def cell(self, row: int, col: int) -> CellValue:
464     if row < 1 or col < 1:
465         return None
466     c = col - 1
467     row_data = self._row_cache.get(row, [])
468     if c >= len(row_data):
469         return None
470     v = row_data[c]
471     if v == "":
472         return None
473     return _coerce_csv_value(v)
474
475 def iter_rows_lazy(self, cols: list[int]) -> Generator[tuple[int, list[CellValue]], None, None]:
476     col_to_pos = {c: i for i, c in enumerate(cols)}
477     for r in range(1, self.max_row + 1):
478         row_data = self._row_cache.get(r, [])
479         vals = [None] * len(cols)
480         for c, pos in col_to_pos.items():
481             row_c = c - 1
482             if 0 <= row_c < len(row_data):
483                 v = row_data[row_c]
484                 vals[pos] = _coerce_csv_value(v) if v != "" else None
485         yield r, vals
486
487 # =====
488 # Загрузки
489 # =====
490
491 def load_sheets(filepath: str, only_sheet: Optional[str] = None) -> list[SheetAdapter]:
492     """Загружает листы из файла.
493
494     ВАЖНО: для .xlsx/.xlsm файлов вызывающий код должен закрыть workbook
495     после использования: `adapters[0].ws.parent.close()` (если adapters не пуст).
496     При использовании через ExcelParser.parse_file() закрытие происходит автоматически.
497     """
498     ext = os.path.splitext(filepath)[1].lower()
499     if ext in (".xlsx", ".xlsm", ".xltx", ".xltm"):
500         return _load_xlsx(filepath, only_sheet)
501     elif ext == ".xls":
502         return _load_xls(filepath, only_sheet)
503     elif ext == ".xlsb":
504         return _load_xlsb(filepath, only_sheet)
505     elif ext == ".csv":
506         return _load_csv(filepath, only_sheet)
507     else:
508         raise ValueError(f"Неподдерживаемый формат: {ext}. "
509                          "Поддерживается: .xlsx .xlsm .xls .xlsb .csv")
510
511
512 def _load_xlsx(filepath: str, only_sheet: Optional[str]) -> list[OpenpyxlAdapter]:
513     ext = os.path.splitext(filepath)[1].lower()
514     if ext == ".xlsm":
515         warnings.warn(
516             f"{os.path.basename(filepath)}: формат .xlsm -- "
517             "файл откроется корректно, VBA-макросы игнорируются.",
518             UserWarning, stacklevel=3,
519         )
520
521     # Проверка формул: один быстрый read_only проход
522     sheets_to_process = list[str] = []
523     try:
524         wb_scan = openpyxl.load_workbook(filepath, data_only=False, read_only=True)
525         try:
526             for sheet_name in wb_scan.sheetnames:
527                 if only_sheet and sheet_name != only_sheet:
528                     continue
529                 ws = wb_scan[sheet_name]
530                 for row in ws.iter_rows(values_only=False):
531                     for cell in row:
532                         if isinstance(cell.value, str) and cell.value.startswith("="):
533                             warnings.warn(
534                                 f"Лист '{sheet_name}': обнаружены формулы. "
535                                 "data_only=True вернёт None для несохранённых значений.",
536                                 UserWarning, stacklevel=3,
537                             )
538                     break
539                 else:
540                     continue
541             break
542

```

```

543         sheets_to_process.append(sheet_name)
544     finally:
545         wb_scan.close()
546 except Exception as e:
547     # FIX v19: упрощённый fallback – без промежуточного открытия файла
548     warnings.warn(
549         f"load_xlsx: не удалось выполнить проверку формул для "
550         f"{os.path.basename(filepath)}: {e}",
551         UserWarning, stacklevel=2,
552     )
553     sheets_to_process = [] # будет заполнен из основного wb ниже
554
555 wb = openpyxl.load_workbook(filepath, data_only=True)
556
557 # Если scan не удался, берём список листов из основного wb
558 if not sheets_to_process:
559     sheets_to_process = [
560         n for n in wb.sheetnames
561         if not only_sheet or n == only_sheet
562     ]
563
564 sheets = []
565 for name in wb.sheetnames:
566     if name not in sheets_to_process:
567         continue
568     ws = wb[name]
569     if not ws.max_row:
570         continue
571     sheets.append(OpenpyxlAdapter(ws, name, wb=wb))
572 # FIX: если не создано ни одного адаптера – закрываем workbook,
573 # иначе он учёт (нет адаптера, через который вызвать close())
574 if not sheets:
575     try:
576         wb.close()
577     except Exception:
578         pass
579 return sheets
580
581
582 def _load_xls(filepath: str, only_sheet: Optional[str]) -> list[XlrdAdapter]:
583     if not HAS_XLRD:
584         raise ImportError("pip install xlrd # для .xls файлов")
585     wb = xlrd.open_workbook(filepath)
586     result = []
587     for name in wb.sheet_names():
588         if only_sheet and name != only_sheet:
589             continue
590         result.append(XlrdAdapter(wb.sheet_by_name(name), wb, name))
591     return result
592
593
594 def _load_xlsb(filepath: str, only_sheet: Optional[str]) -> list[PyxlsbAdapter]:
595     if not HAS_PYXLSB:
596         raise ImportError("pip install pyxlsb # для .xlsb файлов")
597     import pyxlsb as _pyxlsb
598
599     # FIX v19: одно открытие файла – читаем все листы + named ranges за один проход.
600     # Ранее: _load_xlsb открывал файл для списка листов, затем каждый
601     # PyxlsbAdapter.__init__ открывал его повторно (N+1 открытий).
602     result: list[PyxlsbAdapter] = []
603     with _pyxlsb.open_workbook(filepath) as wb:
604         # Собираем named ranges один раз для всех листов
605         all_named: list[dict] = _read_xlsb_named_ranges(wb)
606
607     for sheet_name in wb.sheets:
608         if only_sheet and sheet_name != only_sheet:
609             continue
610
611         # Читаем данные листа
612         tmp: dict[int, dict] = {}
613         max_r = max_c = 0
614         with wb.get_sheet(sheet_name) as ws:
615             for i, row in enumerate(ws.rows()):
616                 r = i + 1
617                 if r > max_r:
618                     max_r = r
619                 for cell in row:
620                     c_idx = cell.c
621                     if c_idx + 1 > max_c:
622                         max_c = c_idx + 1
623                     tmp.setdefault(r, {})[c_idx] = cell.v
624
625         # Конвертируем в row_cache (с конвертацией Excel serial dates)
626         row_cache: dict[int, list] = {}
627         for r, cols in tmp.items():
628             data = [None] * max_c
629             for c_idx, v in cols.items():
630                 if 0 <= c_idx < max_c:
631                     data[c_idx] = _xlsb_convert_date(v) if isinstance(v, float) else v
632             row_cache[r] = data
633

```

```

634         # Фильтруем named ranges для этого листа
635         sheet_named = [nr for nr in all_named if nr["sheet"] == sheet_name]
636
637         result.append(PyxlsbAdapter(
638             sheet_name=sheet_name,
639             row_cache=row_cache,
640             max_row=max_r,
641             max_col=max_c,
642             named_ranges_cache=sheet_named,
643         ))
644
645     return result
646
647
648 def _xlsb_convert_date(v: CellValue) -> CellValue:
649     """Конвертирует Excel serial date number в datetime, если это дата.
650
651     Excel хранит даты как float: целая часть = дни с 1899-12-30,
652     дробная часть = время. Диапазон 1.0..2958465.0 покрывает 1900-01-01..9999-12-31.
653     Целые числа и числа вне диапазона не конвертируются.
654     """
655     if not isinstance(v, float):
656         return v
657     if not math.isfinite(v):
658         return v
659     # Только float с дробной частью или в "типичном" диапазоне дат Excel
660     # Отсекаем отрицательные и слишком большие значения
661     if v < 1.0 or v > 2958465.0:
662         return v
663     # Если это целое число – вероятнее обычное число, а не дата
664     # (даты без времени редки в xlsb, т.к. pyxlsb обычно возвращает их как float с .0)
665     # Проверяем: если дробная часть ненулевая – это дата+время, конвертируем
666     # Если дробная часть нулевая – неоднозначно, оставляем как число
667     if v == int(v):
668         return v
669     try:
670         # Excel epoch: 1899-12-30 (с учётом бага Lotus 1-2-3 с 29 февраля 1900)
671         epoch = datetime.datetime(1899, 12, 30)
672         return epoch + datetime.timedelta(days=v)
673     except (ValueError, OverflowError):
674         return v
675
676
677 def _read_xlsb_named_ranges(wb) -> list[dict]:
678     """Извлекает все named ranges из открытого pyxlsb workbook."""
679     result: list[dict] = []
680     try:
681         candidates = (
682             getattr(wb, "defined_names", None)
683             or getattr(wb, "named_ranges", None)
684             or getattr(wb, "_defined_names", None)
685             or []
686         )
687         for nr in candidates:
688             try:
689                 name = getattr(nr, "name", None) or getattr(nr, "Name", None)
690                 formula = getattr(nr, "formula", None) or getattr(nr, "refers_to", None)
691                 if not name or not formula:
692                     continue
693                 if "!" not in formula:
694                     continue
695                 sheet_part, range_part = formula.split("!", 1)
696                 sheet_name = sheet_part.strip("'\"$")
697                 min_col, min_row, max_col, max_row = _range_boundaries(range_part)
698                 if min_col > max_col or min_row > max_row:
699                     continue
700                 result.append({
701                     "name": name,
702                     "sheet": sheet_name,
703                     "min_row": min_row, "max_row": max_row,
704                     "min_col": min_col, "max_col": max_col,
705                 })
706             except Exception as e:
707                 warnings.warn(
708                     f"xlsb named range: не удалось распарсить: {e}",
709                     UserWarning, stacklevel=2,
710                 )
711     except Exception as e:
712         warnings.warn(
713             f"xlsb: ошибка при чтении named_ranges: {e}",
714             UserWarning, stacklevel=2,
715         )
716     return result
717
718
719 def _detect_encoding(filepath: str) -> str:
720     if HAS_CHARDET:
721         with open(filepath, "rb") as f:
722             raw = f.read(131072) # 128KB для надёжного определения кодировки
723             detected = _chardet.detect(raw)
724             enc = (detected.get("encoding") or "utf-8").strip()

```

```

725     try:
726         codecs.lookup(enc)
727         # FIX: если chardet вернул "utf-8", но файл начинается с BOM,
728         # используем "utf-8-sig" чтобы BOM не попал в данные
729         if enc.lower().replace("-", "") == "utf8" and raw[:3] == b'\xef\xbb\xbf':
730             return "utf-8-sig"
731         return enc
732     except LookupError:
733         pass
734     # FIX: читаем до 1MB – достаточно для надёжного определения кодировки,
735     # но не вызовет MemoryError на огромных CSV.
736     # latin-1 вынесен отдельно – он принимает BCE байты и является последним fallback.
737     _ENC_SAMPLE_SIZE = 1024 * 1024 # 1MB
738     for enc in ("utf-8-sig", "utf-8", "cp1251", "cp1252"):
739         try:
740             with open(filepath, encoding=enc, errors="strict") as f:
741                 while True:
742                     chunk = f.read(_ENC_SAMPLE_SIZE)
743                     if not chunk:
744                         break
745             return enc
746         except (UnicodeDecodeError, LookupError):
747             continue
748     # latin-1 принимает любой байт 0x00-0xFF – гарантированный fallback
749     return "latin-1"
750
751
752 def _load_csv(filepath: str, only_sheet: Optional[str] = None) -> list[CsvAdapter]:
753     sheet_name = os.path.basename(filepath)
754     # FIX: если указан --sheet и имя не совпадает с именем файла, пропускаем
755     if only_sheet and only_sheet != sheet_name:
756         return []
757
758     encoding = _detect_encoding(filepath)
759
760     # FIX v19: пробел убран из разделителей – ложно срабатывает на текстах
761     KNOWN_DELIMITERS = ",;\t|^~"
762     try:
763         with open(filepath, newline="", encoding=encoding) as f:
764             sample = f.read(16384)
765     except:
766         dialect = csv.Sniffer().sniff(sample, delimiters=KNOWN_DELIMITERS)
767         delimiter = dialect.delimiter
768     except csv.Error:
769         candidates = {d: sample.count(d) for d in KNOWN_DELIMITERS}
770         _best = max(candidates, key=candidates.get)
771         delimiter = _best if candidates[_best] > 0 else ","
772
773     return [CsvAdapter(filepath, encoding, delimiter, sheet_name)]
774 except Exception as e:
775     raise ValueError(f"Не удалось прочитать CSV '{filepath}': {e}") from e
776
777
778 # =====
779 # Утилиты значений
780 # =====
781
782 def _to_str(v: CellValue) -> str:
783     return "" if v is None else str(v).strip()
784
785
786 def _is_empty(v: CellValue) -> bool:
787     return _to_str(v) == ""
788
789
790 def _is_numeric(v: CellValue) -> bool:
791     if isinstance(v, bool):
792         return False
793     if isinstance(v, (int, float)):
794         return math.isfinite(v)
795     s = _to_str(v).replace(",", ".").replace(" ", "").replace("%", "")
796     if not s:
797         return False
798     try:
799         return math.isfinite(float(s))
800     except ValueError:
801         return False
802
803
804 def _is_date(v: CellValue) -> bool:
805     return isinstance(v, (datetime.datetime, datetime.date))
806
807
808 def _is_year(v: CellValue) -> bool:
809     if isinstance(v, bool):
810         return False
811     if isinstance(v, float):
812         if not math.isfinite(v):
813             return False
814         if v != int(v):
815             return False

```

```

816         return 1900 <= int(v) <= 2200
817     if isinstance(v, int):
818         return 1900 <= v <= 2200
819     s = _to_str(v)
820     try:
821         return 1900 <= int(s) <= 2200
822     except ValueError:
823         return False
824
825
826 def _serialize(v: CellValue) -> Any:
827     # FIX v19: обработка time и timedelta
828     # FIX v20: обработка NaN/Infinity – невалидны в JSON
829     if isinstance(v, float) and (math.isnan(v) or math.isinf(v)):
830         return None
831     if isinstance(v, (datetime.datetime, datetime.date)):
832         return v.isoformat()
833     if isinstance(v, datetime.time):
834         return v.isoformat()
835     if isinstance(v, datetime.timedelta):
836         total_sec = v.total_seconds()
837         sign = "-" if total_sec < 0 else ""
838         abs_total = int(abs(total_sec))
839         hours, remainder = divmod(abs_total, 3600)
840         minutes, seconds = divmod(remainder, 60)
841         return f"{sign}{hours:02d}:{minutes:02d}:{seconds:02d}"
842     return v
843
844
845 def _detect_dtype(values: list[CellValue]) -> str:
846     non_empty = [v for v in values if not _is_empty(v)]
847     if not non_empty:
848         return "text"
849     total = len(non_empty)
850
851     # FIX v19: распознавание boolean
852     # FIX v20: >= 0.5 вместо > 0.5, чтобы 50% bool + 50% num не возвращал "text"
853     bools = sum(1 for v in non_empty if isinstance(v, bool))
854     if bools / total >= 0.5:
855         return "boolean"
856
857     nums = sum(1 for v in non_empty if _is_numeric(v))
858     dates = sum(1 for v in non_empty if _is_date(v))
859     if dates / total > 0.5:
860         return "date"
861     if nums / total > 0.7:
862         pct_count = sum(
863             1 for v in non_empty
864             if isinstance(v, str) and "%" in v
865         )
866         return "percent" if pct_count / total > 0.3 else "number"
867     return "text"
868
869
870 # =====
871 # Score-based детектор заголовков
872 # =====
873
874 def _score_header_row(vals: list[CellValue]) -> float:
875     non_empty = [v for v in vals if not _is_empty(v)]
876     if not non_empty:
877         return 0.0
878
879     # Паттерн "числа-дни" (1,2,3...31 подряд)
880     numeric_vals = [v for v in non_empty if _is_numeric(v) and not _is_year(v)]
881     is_day_header = False
882     if len(numeric_vals) >= 5:
883         day_numbers = []
884         for v in numeric_vals:
885             if isinstance(v, bool):
886                 continue
887             if isinstance(v, float):
888                 if not math.isfinite(v):
889                     continue
890                 n = int(v)
891             elif isinstance(v, int):
892                 n = v
893             elif isinstance(v, str):
894                 # FIX v19: CSV возвращает числа как строки
895                 try:
896                     n = int(float(v.replace(",", ".").replace(" ", "").replace("%", "")))
897                 except (ValueError, OverflowError):
898                     continue
899             else:
900                 continue
901             if 1 <= n <= 31:
902                 day_numbers.append(n)
903         if len(day_numbers) >= 5:
904             day_numbers_sorted = sorted(day_numbers)
905             consecutive = sum(
906                 1 for i in range(len(day_numbers_sorted) - 1)

```

```

907         if 1 <= (day_numbers_sorted[i + 1] - day_numbers_sorted[i]) <= 2
908     )
909     if consecutive >= len(day_numbers_sorted) * 0.6:
910         is_day_header = True
911
912 text_like = 0.0
913 for v in non_empty:
914     if _is_year(v):
915         if isinstance(v, str):
916             text_like += 0.3
917     elif _is_date(v):
918         text_like += 0.5
919     elif _is_numeric(v):
920         if is_day_header:
921             text_like += 0.5
922         else:
923             text_like -= 0.2
924     else:
925         s = _to_str(v)
926         if len(s) == 1:
927             text_like += 0.7
928         elif 2 <= len(s) <= 60:
929             text_like += 1.0
930         elif len(s) > 60:
931             text_like -= 0.5
932 ratio = text_like / len(non_empty)
933 if len(non_empty) < 2:
934     ratio *= 0.6
935 return max(0.0, min(1.0, ratio))
936
937
938 def _is_header_row(vals: list[CellValue], threshold: float = 0.4) -> bool:
939     return _score_header_row(vals) >= threshold
940
941
942 def _dedupe_headers(names: list[str]) -> list[str]:
943     seen: dict[str, int] = {}
944     result: list[str] = []
945     for i, name in enumerate(names):
946         key = name if name else f"_col_{i + 1}"
947         if key not in seen:
948             seen[key] = 1
949             result.append(key)
950         else:
951             seen[key] += 1
952             result.append(f"{key}_{seen[key]}")
953     return result
954
955
956 # =====
957 # Поточковый writer
958 # =====
959
960 class StreamingWriter:
961     def __init__(self, output_path: str, fmt: str, file_meta: dict):
962         self.fmt = fmt
963         self.output_path = output_path
964         self._table_count = 0
965         self._row_count = 0
966         self._file_meta = file_meta
967         self._json_fh = None
968         self._jsonl_fh = None
969         self._csv_dir = None
970         self._open(output_path, fmt)
971
972     def _open(self, path: str, fmt: str) -> None:
973         if fmt == "json":
974             self._json_fh = open(path, "w", encoding="utf-8")
975             # FIX: сначала пишем все ключи meta, затем tables_data -
976             # гарантированно не теряем ключи, даже если порядок dict изменится.
977             self._json_fh.write("{")
978             first = True
979             for k, v in self._file_meta.items():
980                 if not first:
981                     self._json_fh.write(", ")
982                 first = False
983                 self._json_fh.write(
984                     json.dumps(k, ensure_ascii=False) + ": "
985                     + json.dumps(v, ensure_ascii=False)
986                 )
987             if not first:
988                 self._json_fh.write(", ")
989             self._json_fh.write("tables_data": [\n')
990             self._first_table = True
991         elif fmt == "jsonl":
992             self._jsonl_fh = open(path, "w", encoding="utf-8")
993         elif fmt == "csv":
994             os.makedirs(path, exist_ok=True)
995             self._csv_dir = path
996
997     def write_table(self, table: dict) -> None:

```

```

998 self._table_count += 1
999 self._row_count += len(table.get("rows", []))
1000 if self.fmt == "json":
1001     sep = "" if self._first_table else ",\n"
1002     self._first_table = False
1003     meta = {k: v for k, v in table.items() if k != "rows"}
1004     # Формируем JSON объект без хака с обрезкой последнего символа
1005     self._json_fh.write(sep + "{")
1006     meta_items = list(meta.items())
1007     for idx, (k, v) in enumerate(meta_items):
1008         if idx > 0:
1009             self._json_fh.write(", ")
1010         self._json_fh.write(
1011             json.dumps(k, ensure_ascii=False) + ": "
1012             + json.dumps(v, ensure_ascii=False, default=str)
1013         )
1014     if meta_items:
1015         self._json_fh.write(", ")
1016     self._json_fh.write("rows": [')
1017     rows = table.get("rows", [])
1018     for i, row in enumerate(rows):
1019         row_sep = "" if i == 0 else ", "
1020         self._json_fh.write(
1021             row_sep + json.dumps(row, ensure_ascii=False, default=str)
1022         )
1023     self._json_fh.write("]}")
1024 elif self.fmt == "jsonl":
1025     for row in table.get("rows", []):
1026         record = {"_sheet": table["sheet"], "_table": table["name"], **row}
1027         self._jsonl_fh.write(
1028             json.dumps(record, ensure_ascii=False, default=str) + "\n"
1029         )
1030 elif self.fmt == "csv":
1031     rows = table.get("rows", [])
1032     if not rows:
1033         return
1034     safe = (
1035         table["name"]
1036         .replace("/", "_")
1037         .replace("\\", "\\")
1038         .replace(":", "_")
1039         .replace(" ", "_")[:60]
1040     )
1041     path = os.path.join(self._csv_dir,
1042                         f"{self._table_count:02d}_{safe}.csv")
1043     columns = table.get("columns")
1044     if columns:
1045         fieldnames = [col["name"] for col in columns]
1046     else:
1047         fieldnames = list(dict.fromkeys(
1048             k for row in rows for k in row
1049         ))
1050     with open(path, "w", newline="", encoding="utf-8-sig") as f:
1051         w = csv.DictWriter(f, fieldnames=fieldnames, extrasaction="ignore")
1052         w.writeheader()
1053         w.writerows(rows)
1054
1055 def close(self) -> None:
1056     if self.fmt == "json" and self._json_fh:
1057         self._json_fh.write("\n]")
1058         self._json_fh.close()
1059         self._json_fh = None
1060     elif self.fmt == "jsonl" and self._jsonl_fh:
1061         self._jsonl_fh.close()
1062         self._jsonl_fh = None
1063
1064 def __enter__(self):
1065     return self
1066
1067 def __exit__(self, exc_type, exc_val, exc_tb):
1068     if exc_type is None:
1069         # Парсинг прошёл без ошибок — финализируем файл.
1070         # Если close() упадёт (диск, права), ошибка пробросится наружу.
1071         self.close()
1072     else:
1073         # FIX v20: при ошибке парсинга — закрываем файлы и удаляем
1074         # битый output, чтобы не оставлять невалидные файлы на диске.
1075         try:
1076             if self._json_fh:
1077                 self._json_fh.close()
1078                 self._json_fh = None
1079             if self._jsonl_fh:
1080                 self._jsonl_fh.close()
1081                 self._jsonl_fh = None
1082         except Exception:
1083             pass
1084         # Удаляем повреждённый output
1085         try:
1086             if self.output_path:
1087                 if self.fmt in ("json", "jsonl") and os.path.isfile(self.output_path):
1088                     os.remove(self.output_path)
1089                     warnings.warn(
1090                         f"Удалён неполный файл '{self.output_path}' "

```

```

1089         "после ошибки парсинга.",
1090         UserWarning, stacklevel=2,
1091     )
1092     elif self.fmt == "csv" and self._csv_dir and os.path.isdir(self._csv_dir):
1093         # FIX: удаляем частично записанную CSV-директорию
1094         import shutil
1095         shutil.rmtree(self._csv_dir, ignore_errors=True)
1096         warnings.warn(
1097             f"Удалена неполная директория '{self._csv_dir}' "
1098             "после ошибки парсинга.",
1099             UserWarning, stacklevel=2,
1100         )
1101     except OSError:
1102         pass
1103     return False
1104
1105 @property
1106 def stats(self) -> tuple[int, int]:
1107     return self._table_count, self._row_count
1108
1109 # =====
1110 # Парсер
1111 # =====
1112
1113 class ExcelParser:
1114     def __init__(
1115         self,
1116         header_threshold: float = 0.4,
1117         skip_hidden: bool = False,
1118         min_data_cells: int = 2,
1119         max_empty_streak: int = 50,
1120     ):
1121         # FIX v19: валидация параметров
1122         if not 0.0 <= header_threshold <= 1.0:
1123             raise ValueError(f"header_threshold должен быть в [0.0, 1.0], получено: {header_threshold}")
1124         if min_data_cells < 1:
1125             raise ValueError(f"min_data_cells должен быть >= 1, получено: {min_data_cells}")
1126         if max_empty_streak < 1:
1127             raise ValueError(f"max_empty_streak должен быть >= 1, получено: {max_empty_streak}")
1128
1129         self.header_threshold = header_threshold
1130         self.skip_hidden = skip_hidden
1131         self.min_data_cells = min_data_cells
1132         self.max_empty_streak = max_empty_streak
1133         self._vis_cache: dict[tuple[int, str], tuple[list[int], list[int]]] = {}
1134
1135     # -- Видимые строки / колонки -----
1136
1137     def _visible(self, adapter: SheetAdapter) -> tuple[list[int], list[int]]:
1138         # FIX: используем (id, name) чтобы избежать коллизий после GC
1139         key = (id(adapter), adapter.name)
1140         if key not in self._vis_cache:
1141             hr = adapter.hidden_rows() if self.skip_hidden else set()
1142             hc = adapter.hidden_cols() if self.skip_hidden else set()
1143             rows = [r for r in range(1, adapter.max_row + 1) if r not in hr]
1144             cols = [c for c in range(1, adapter.max_col + 1) if c not in hc]
1145             self._vis_cache[key] = (rows, cols)
1146         return self._vis_cache[key]
1147
1148     # -- Общий метод сборки строк -----
1149
1150     def _build_rows(
1151         self,
1152         adapter: SheetAdapter,
1153         data_rows: list[int],
1154         col_map: dict[int, str],
1155     ) -> tuple[list[Row], dict[int, list[CellValue]]]:
1156         """
1157         Читает ячейки один раз, возвращает (rows_out, col_values).
1158         col_values используется для _detect_dtype без повторного чтения.
1159         """
1160         col_values: dict[int, list[CellValue]] = {c: [] for c in col_map}
1161         rows_out: list[Row] = []
1162         for r in data_rows:
1163             rd: dict[str, Any] = {}
1164             has = False
1165             for c, col_name in col_map.items():
1166                 v = adapter.cell(r, c)
1167                 col_values[c].append(v)
1168                 if not _is_empty(v):
1169                     sv = _serialize(v)
1170                     if sv is not None:
1171                         rd[col_name] = sv
1172                     has = True
1173             if has:
1174                 rows_out.append(rd)
1175         return rows_out, col_values
1176
1177     # -- Парсинг диапазона -----
1178
1179

```

```

1180     def _parse_range(
1181         self,
1182         adapter: SheetAdapter,
1183         min_row: int, max_row: int,
1184         min_col: int, max_col: int,
1185         source: str,
1186         name: str,
1187     ) -> Optional[dict]:
1188         if min_col > max_col or min_row > max_row:
1189             return None
1190
1191         vis_rows, vis_cols = self._visible(adapter)
1192         vis_cols_set = set(vis_cols)
1193         cols = [c for c in range(min_col, max_col + 1) if c in vis_cols_set]
1194         if not cols:
1195             return None
1196         rows_in = [r for r in vis_rows if min_row <= r <= max_row]
1197         if len(rows_in) < 2:
1198             return None
1199
1200         header_row = rows_in[0]
1201         data_rows = rows_in[1:]
1202
1203         raw_headers = [_to_str(adapter.cell(header_row, c)) for c in cols]
1204         deduped = _dedupe_headers(raw_headers)
1205         header_dict: dict[int, str] = {
1206             c: deduped[i] for i, c in enumerate(cols) if deduped[i]
1207         }
1208
1209         # Fallback для колонок без заголовка
1210         col_map: dict[int, str] = {}
1211         for c in cols:
1212             col_map[c] = header_dict.get(c, get_column_letter(c))
1213
1214         rows_out, col_values = self._build_rows(adapter, data_rows, col_map)
1215
1216         if not rows_out:
1217             return None
1218
1219         return {
1220             "sheet": adapter.name,
1221             "name": name,
1222             "source": source,
1223             "header_row": header_row,
1224             "data_start": data_rows[0],
1225             "data_end": data_rows[-1],
1226             "columns": [
1227                 {
1228                     "letter": get_column_letter(c),
1229                     "name": col_map[c],
1230                     "type": _detect_dtype(col_values[c]),
1231                 }
1232                 for c in cols
1233             ],
1234             "rows": rows_out,
1235         }
1236
1237     # -- Источник 1: встроенные таблицы -----
1238
1239     def _extract_native_tables(self, adapter: SheetAdapter) -> list[dict]:
1240         results = []
1241         for tbl in adapter.native_tables():
1242             p = self._parse_range(
1243                 adapter,
1244                 tbl["min_row"], tbl["max_row"],
1245                 tbl["min_col"], tbl["max_col"],
1246                 TABLE_SOURCE_NATIVE, tbl["name"],
1247             )
1248             if p:
1249                 results.append(p)
1250         return results
1251
1252     # -- Источник 2: именованные диапазоны -----
1253
1254     def _extract_named_ranges_from_wb(self, wb, adapters_map: dict[str, SheetAdapter]) -> list[dict]:
1255         results: list[dict] = []
1256         try:
1257             defined = wb.defined_names
1258         except AttributeError:
1259             return results
1260
1261         for dn in defined:
1262             try:
1263                 destinations = dn.destinations
1264             except AttributeError:
1265                 continue
1266             if isinstance(destinations, str):
1267                 if "!" not in destinations:
1268                     continue
1269                 sheet_part, range_part = destinations.split("!", 1)
1270                 sheet_title = sheet_part.strip("'\"$")

```

```

1271         try:
1272             min_col, min_row, max_col, max_row = _range_boundaries(range_part)
1273         except Exception as e:
1274             warnings.warn(
1275                 f"Именованный диапазон '{dn.name}': "
1276                 f"не удалось распарсить диапазон '{range_part}': {e}",
1277                 UserWarning, stacklevel=2,
1278             )
1279             continue
1280         if sheet_title not in adapters_map:
1281             continue
1282         adapter = adapters_map[sheet_title]
1283         p = self._parse_range(
1284             adapter, min_row, max_row, min_col, max_col,
1285             TABLE_SOURCE_NAMED, dn.name,
1286         )
1287         if p:
1288             results.append(p)
1289     else:
1290         for sheet_title, ref in destinations:
1291             if sheet_title not in adapters_map:
1292                 continue
1293             try:
1294                 min_col, min_row, max_col, max_row = _range_boundaries(ref)
1295             except Exception as e:
1296                 warnings.warn(
1297                     f"Именованный диапазон '{dn.name}' на листе '{sheet_title}': "
1298                     f"не удалось распарсить диапазон '{ref}': {e}",
1299                     UserWarning, stacklevel=2,
1300                 )
1301                 continue
1302             adapter = adapters_map[sheet_title]
1303             p = self._parse_range(
1304                 adapter, min_row, max_row, min_col, max_col,
1305                 TABLE_SOURCE_NAMED, dn.name,
1306             )
1307             if p:
1308                 results.append(p)
1309     return results
1310
1311 def _extract_named_ranges_from_adapter(self, adapter: SheetAdapter) -> list[dict]:
1312     results: list[dict] = []
1313     for nr in adapter.named_ranges():
1314         p = self._parse_range(
1315             adapter,
1316             nr["min_row"], nr["max_row"],
1317             nr["min_col"], nr["max_col"],
1318             TABLE_SOURCE_NAMED, nr["name"],
1319         )
1320         if p:
1321             results.append(p)
1322     return results
1323
1324 # -- Источник 3: эвристика -----
1325
1326 def _extract_heuristic(
1327     self, adapter: SheetAdapter, external_used_rows: set[int]
1328 ) -> list[dict]:
1329     vis_rows, vis_cols = self._visible(adapter)
1330     if not vis_rows or not vis_cols:
1331         return []
1332
1333     row_idx: dict[int, int] = {r: i for i, r in enumerate(vis_rows)}
1334     tables: list[dict] = []
1335     used_rows: set[int] = set(external_used_rows)
1336     table_counter = 0
1337     i = 0
1338
1339     while i < len(vis_rows):
1340         row = vis_rows[i]
1341         vals = [adapter.cell(row, c) for c in vis_cols]
1342
1343         if all(_is_empty(v) for v in vals):
1344             i += 1
1345             continue
1346
1347         if not _is_header_row(vals, self.header_threshold):
1348             i += 1
1349             continue
1350
1351         # Многострочный заголовок
1352         header_rows = [row]
1353         header_dict: dict[int, str] = {}
1354         for c in vis_cols:
1355             h = _to_str(adapter.cell(row, c))
1356             if h:
1357                 header_dict[c] = h
1358
1359         for j in range(i + 1, min(i + 4, len(vis_rows))):
1360             r2 = vis_rows[j]
1361             r2v = [adapter.cell(r2, c) for c in vis_cols]

```

```

1362         sc = _score_header_row(r2v)
1363         nums = sum(1 for v in r2v if _is_numeric(v) and not _is_year(v))
1364         if sc >= 0.35 and nums == 0:
1365             header_rows.append(r2)
1366             for c in vis_cols:
1367                 h = _to_str(adapter.cell(r2, c))
1368                 if h:
1369                     header_dict[c] = (
1370                         header_dict[c] + " / " + h
1371                         if c in header_dict else h
1372                     )
1373             else:
1374                 break
1375
1376         active_cols = set(header_dict.keys())
1377         if not active_cols:
1378             i += 1
1379             continue
1380
1381         start_i = row_idx.get(header_rows[-1], i) + 1
1382
1383         # Строки данных
1384         data_rows: list[int] = []
1385         empty_streak = 0
1386
1387         for di in range(start_i, len(vis_rows)):
1388             r = vis_rows[di]
1389             rv = [adapter.cell(r, c) for c in active_cols]
1390             non_e = sum(1 for v in rv if not _is_empty(v))
1391
1392             if non_e == 0:
1393                 empty_streak += 1
1394                 if empty_streak >= self.max_empty_streak:
1395                     break
1396                 continue
1397
1398             # FIX v20: строки ниже min_data_cells не сбрасывают empty_streak –
1399             # иначе одиночные ячейки между таблицами мешают обнаружению границ
1400             if non_e < self.min_data_cells:
1401                 empty_streak += 1
1402                 if empty_streak >= self.max_empty_streak:
1403                     break
1404                 continue
1405
1406             # FIX: если после пустых строк встречается строка, похожая на
1407             # заголовок новой таблицы – прекращаем сбор текущей таблицы.
1408             # Читаем ВСЕ видимые колонки (не только active_cols), т.к. новая
1409             # таблица может иметь другой набор колонок.
1410             if empty_streak > 0 and data_rows:
1411                 full_rv = [adapter.cell(r, c) for c in vis_cols]
1412                 if _is_header_row(full_rv, self.header_threshold):
1413                     break
1414
1415             empty_streak = 0
1416             data_rows.append(r)
1417
1418         if not data_rows:
1419             i += 1
1420             continue
1421
1422         # FIX v20: проверяем пересечение только по реальным строкам
1423         # (заголовки + данные), а не сплошным диапазоном
1424         actual_rows = set(header_rows) | set(data_rows)
1425         if actual_rows & used_rows:
1426             i += 1
1427             continue
1428
1429         # Сборка
1430         sorted_cols = sorted(active_cols)
1431         raw_col_names = [header_dict[c] for c in sorted_cols]
1432         deduped_names = _dedupe_headers(raw_col_names)
1433
1434         col_map: dict[int, str] = {}
1435         for c, col_name in zip(sorted_cols, deduped_names):
1436             col_map[c] = col_name
1437
1438         # FIX v19: используем _build_rows для единого чтения ячеек
1439         rows_out, col_values = self._build_rows(adapter, data_rows, col_map)
1440
1441         if rows_out:
1442             table_counter += 1
1443             tables.append({
1444                 "sheet": adapter.name,
1445                 "name": f"{adapter.name} / Таблица {table_counter}",
1446                 "source": TABLE_SOURCE_HEURISTIC,
1447                 "header_row": header_rows[0],
1448                 "data_start": data_rows[0],
1449                 "data_end": data_rows[-1],
1450                 "columns": [
1451                     {
1452                         "letter": get_column_letter(c),

```

```

1453         "name": col_map[c],
1454         "type": _detect_dtype(col_values[c]),
1455     }
1456     for c in sorted_cols
1457 ],
1458     "rows": rows_out,
1459 })
1460 # FIX v20: блокируем только реальные строки, не сплошной диапазон
1461 used_rows.update(actual_rows)
1462
1463 i = row_idx.get(data_rows[-1], i) + 1
1464
1465 return tables
1466
1467 # -- Источник 4: вертикальные таблицы -----
1468
1469 def _extract_vertical(self, adapter: SheetAdapter, used_rows: set[int]) -> list[dict]:
1470     vis_rows, vis_cols = self._visible(adapter)
1471     if not vis_rows or len(vis_cols) < 2:
1472         return []
1473
1474     col_a = vis_cols[0]
1475     other_c = vis_cols[1:]
1476
1477     blocks: list[list[int]] = []
1478     current: list[int] = []
1479     for r in vis_rows:
1480         rv = [adapter.cell(r, c) for c in vis_cols]
1481         if all(_is_empty(v) for v in rv):
1482             if current:
1483                 blocks.append(current)
1484                 current = []
1485             else:
1486                 current.append(r)
1487         if current:
1488             blocks.append(current)
1489
1490     results: list[dict] = []
1491
1492     for block in blocks:
1493         if len(block) < 2:
1494             continue
1495
1496         header_row = block[0]
1497
1498         if header_row in used_rows:
1499             continue
1500
1501         header_vals = [adapter.cell(header_row, c) for c in vis_cols]
1502         if not _is_header_row(header_vals, 0.3):
1503             continue
1504
1505         data_candidates = block[1:]
1506
1507         text_in_a = 0
1508         numeric_in_rest = 0
1509         total_rest = 0
1510         for r in data_candidates:
1511             if r in used_rows:
1512                 continue
1513             va = adapter.cell(r, col_a)
1514             if not _is_empty(va) and not _is_numeric(va):
1515                 text_in_a += 1
1516             for c in other_c:
1517                 v = adapter.cell(r, c)
1518                 if not _is_empty(v):
1519                     total_rest += 1
1520                     if _is_numeric(v):
1521                         numeric_in_rest += 1
1522
1523         if text_in_a < 2:
1524             continue
1525         if total_rest == 0 or numeric_in_rest / total_rest < 0.5:
1526             continue
1527
1528         data_rows = [r for r in data_candidates if r not in used_rows]
1529         if not data_rows:
1530             continue
1531
1532         raw_headers = [_to_str(adapter.cell(header_row, c)) for c in vis_cols]
1533         deduped_hdr = _dedupe_headers(raw_headers)
1534         col_map: dict[int, str] = {c: deduped_hdr[i] for i, c in enumerate(vis_cols)}
1535
1536         # FIX v19: используем _build_rows
1537         rows_out, col_values = self._build_rows(adapter, data_rows, col_map)
1538
1539         if not rows_out:
1540             continue
1541
1542         tbl_num = len(results) + 1
1543         results.append({

```

```

1544         "sheet": adapter.name,
1545         "name": f"{adapter.name} / Вертикальная таблица {tbl_num}",
1546         "source": TABLE_SOURCE_VERTICAL,
1547         "header_row": header_row,
1548         "data_start": data_rows[0],
1549         "data_end": data_rows[-1],
1550         "columns": [
1551             {
1552                 "letter": get_column_letter(c),
1553                 "name": col_map[c],
1554                 "type": _detect_dtype(col_values[c]),
1555             }
1556             for c in vis_cols
1557         ],
1558         "rows": rows_out,
1559     })
1560     # FIX v20: блокируем только реальные строки (заголовки + данные)
1561     used_rows.add(header_row)
1562     used_rows.update(data_rows)
1563
1564     return results
1565
1566     # -- Источник 5: таблицы без заголовка -----
1567
1568     def _extract_headerless(self, adapter: SheetAdapter, used_rows: set[int]) -> list[dict]:
1569         vis_rows, vis_cols = self._visible(adapter)
1570         if not vis_rows or not vis_cols:
1571             return []
1572
1573         tables: list[dict] = []
1574         table_counter = 0
1575         i = 0
1576
1577         while i < len(vis_rows):
1578             r = vis_rows[i]
1579             vals = [adapter.cell(r, c) for c in vis_cols]
1580
1581             if all(_is_empty(v) for v in vals):
1582                 i += 1
1583                 continue
1584
1585             if r in used_rows:
1586                 i += 1
1587                 continue
1588
1589             block_rows: list[int] = []
1590             empty_streak = 0
1591             # FIX: отслеживаем реальную позицию j для корректного продвижения i
1592             scan_end = i
1593             for j in range(i, len(vis_rows)):
1594                 scan_end = j
1595                 rj = vis_rows[j]
1596                 if rj in used_rows:
1597                     break
1598                 rv = [adapter.cell(rj, c) for c in vis_cols]
1599                 non_e = sum(1 for v in rv if not _is_empty(v))
1600                 if non_e == 0:
1601                     empty_streak += 1
1602                     if empty_streak >= self.max_empty_streak:
1603                         break
1604                     continue
1605                 empty_streak = 0
1606                 block_rows.append(rj)
1607
1608             if len(block_rows) < 2:
1609                 i = scan_end + 1
1610                 continue
1611
1612             col_map: dict[int, str] = {c: get_column_letter(c) for c in vis_cols}
1613
1614             # FIX v19: используем _build_rows
1615             rows_out, col_values = self._build_rows(adapter, block_rows, col_map)
1616
1617             if rows_out:
1618                 table_counter += 1
1619                 tables.append({
1620                     "sheet": adapter.name,
1621                     "name": f"{adapter.name} / Беззаголовочная {table_counter}",
1622                     "source": TABLE_SOURCE_HEADERLESS,
1623                     "header_row": block_rows[0],
1624                     "data_start": block_rows[0],
1625                     "data_end": block_rows[-1],
1626                     "columns": [
1627                         {
1628                             "letter": get_column_letter(c),
1629                             "name": col_map[c],
1630                             "type": _detect_dtype(col_values[c]),
1631                         }
1632                         for c in vis_cols
1633                     ],
1634                     "rows": rows_out,

```

```

1635         })
1636         # FIX v20: блокируем только реальные строки
1637         used_rows.update(block_rows)
1638
1639         i = scan_end + 1
1640
1641         return tables
1642
1643     # -- parse_sheet -----
1644
1645     def parse_sheet(
1646         self,
1647         adapter: SheetAdapter,
1648         wb=None,
1649         all_adapters: Optional[dict[str, SheetAdapter]] = None,
1650     ) -> list[dict]:
1651         self._vis_cache.clear()
1652
1653         all_tables: list[dict] = []
1654         used_rows: set[int] = set()
1655
1656         # 1. Встроенные таблицы
1657         native = self._extract_native_tables(adapter)
1658         for t in native:
1659             used_rows.update(range(t["header_row"], t["data_end"] + 1))
1660         all_tables.extend(native)
1661
1662         # 2. Именованные диапазоны (ищем всегда)
1663         named: list[dict] = []
1664         if wb is not None and all_adapters:
1665             named = [t for t in self._extract_named_ranges_from_wb(wb, all_adapters)
1666                      if t["sheet"] == adapter.name]
1667         else:
1668             named = self._extract_named_ranges_from_adapter(adapter)
1669
1670         for t in named:
1671             span = set(range(t["header_row"], t["data_end"] + 1))
1672             if not (span & used_rows):
1673                 used_rows.update(span)
1674                 all_tables.append(t)
1675
1676         # 3. Эвристика - FIX v19: запускается ВСЕГДА, не только при пустом all_tables
1677         heuristic = self._extract_heuristic(adapter, used_rows)
1678         for t in heuristic:
1679             used_rows.update(range(t["header_row"], t["data_end"] + 1))
1680         all_tables.extend(heuristic)
1681
1682         # 4. Вертикальные таблицы
1683         vertical = self._extract_vertical(adapter, used_rows)
1684         for t in vertical:
1685             used_rows.update(range(t["header_row"], t["data_end"] + 1))
1686         all_tables.extend(vertical)
1687
1688         # 5. Беззаголовочные таблицы
1689         all_tables.extend(self._extract_headerless(adapter, used_rows))
1690
1691         return all_tables
1692
1693     # -- parse_file -----
1694
1695     def parse_file(
1696         self,
1697         filepath: str,
1698         output_path: Optional[str] = None,
1699         fmt: str = "json",
1700         only_sheet: Optional[str] = None,
1701         streaming: bool = False,
1702     ) -> dict:
1703         print(f"\n{'=' * 72}")
1704         print(f"  {os.path.basename(filepath)}")
1705         print(f"{'=' * 72}")
1706
1707         ext = os.path.splitext(filepath)[1].lower()
1708         adapters = load_sheets(filepath, only_sheet)
1709
1710         wb = None
1711         if ext in (".xlsx", ".xlsm", ".xltx", ".xltm"):
1712             first_opx = next(
1713                 (a for a in adapters if isinstance(a, OpenpyxlAdapter)), None
1714             )
1715             if first_opx is not None:
1716                 wb = first_opx._ws.parent
1717         adapters_map = {a.name: a for a in adapters}
1718
1719         writer: Optional[StreamingWriter] = None
1720         if streaming and output_path:
1721             file_meta = {"file": os.path.basename(filepath), "format": ext.lstrip(".")}
1722             writer = StreamingWriter(output_path, fmt, file_meta)
1723         elif streaming and not output_path:
1724             warnings.warn(
1725                 "--stream требует --out-dir или явного output_path. "

```

```

1726         "Данные не будут сохранены на диск.",
1727         UserWarning, stacklevel=2,
1728     )
1729
1730     all_tables: list[dict] = []
1731     stream_sources: dict[str, int] = {
1732         TABLE_SOURCE_NATIVE: 0,
1733         TABLE_SOURCE_NAMED: 0,
1734         TABLE_SOURCE_HEURISTIC: 0,
1735         TABLE_SOURCE_VERTICAL: 0,
1736         TABLE_SOURCE_HEADERLESS: 0,
1737     }
1738     iter_list = (
1739         tqdm(adapters, desc="Листы", unit="лист")
1740         if HAS_TQDM and len(adapters) > 3
1741         else adapters
1742     )
1743
1744     parse_error = None
1745     try:
1746         for adapter in iter_list:
1747             tables = self.parse_sheet(adapter, wb=wb, all_adapters=adapters_map)
1748
1749             src_counts: dict[str, int] = {}
1750             for t in tables:
1751                 src = t["source"]
1752                 src_counts[src] = src_counts.get(src, 0) + 1
1753                 if src in stream_sources:
1754                     stream_sources[src] += 1
1755                 if writer:
1756                     writer.write_table(t)
1757
1758             summary = ", ".join(f"{k}:{v}" for k, v in src_counts.items()) or "-"
1759             print(f" '{adapter.name}' "
1760                   f"({adapter.max_row}x{adapter.max_col}) "
1761                   f"-> {len(tables)} таблиц [{summary}]")
1762
1763             if not streaming:
1764                 all_tables.extend(tables)
1765     except Exception as e:
1766         parse_error = e
1767     finally:
1768         # Закрываем writer: при ошибке парсинга — без финализации;
1769         # при успехе — полный close() с пробросом ошибок записи.
1770         close_error = None
1771         if writer:
1772             if parse_error is not None:
1773                 try:
1774                     if writer._json_fh:
1775                         writer._json_fh.close()
1776                         writer._json_fh = None
1777                     if writer._jsonl_fh:
1778                         writer._jsonl_fh.close()
1779                         writer._jsonl_fh = None
1780                     # FIX v20: удаляем битый output при ошибке парсинга
1781                     if output_path and fmt in ("json", "jsonl"):
1782                         if os.path.isfile(output_path):
1783                             os.remove(output_path)
1784                     elif output_path and fmt == "csv" and writer._csv_dir:
1785                         if os.path.isdir(writer._csv_dir):
1786                             import shutil
1787                             shutil.rmtree(writer._csv_dir, ignore_errors=True)
1788                 except Exception:
1789                     pass
1790             else:
1791                 try:
1792                     writer.close()
1793                 except Exception as e:
1794                     close_error = e
1795         # Закрываем openpyxl workbook после парсинга
1796         if wb is not None:
1797             try:
1798                 wb.close()
1799             except Exception:
1800                 pass
1801
1802     if parse_error is not None:
1803         if close_error is not None:
1804             warnings.warn(
1805                 f"Дополнительная ошибка при закрытии writer: {close_error}",
1806                 UserWarning, stacklevel=2,
1807             )
1808         raise parse_error
1809     if close_error is not None:
1810         raise close_error
1811
1812     if writer:
1813         n_tables, n_rows = writer.stats
1814     else:
1815         n_tables = len(all_tables)
1816         n_rows = sum(len(t["rows"]) for t in all_tables)

```

```

1817 result = {
1818     "file": os.path.basename(filepath),
1819     "format": ext.lstrip("."),
1820     "sheets": len(adapters),
1821     "tables": n_tables,
1822     "total_rows": n_rows,
1823     "sources": stream_sources,
1824     "tables_data": all_tables,
1825 }
1826
1827
1828 if output_path and not streaming:
1829     _write_output(result, output_path, fmt)
1830     print(f" -> {output_path}")
1831
1832
1833 print(f"\n{'=' * 72}")
1834 print(
1835     f" {n_tables} таблиц | {n_rows} строк | "
1836     f"native:{result['sources'][TABLE_SOURCE_NATIVE]} "
1837     f"named:{result['sources'][TABLE_SOURCE_NAMED]} "
1838     f"heuristic:{result['sources'][TABLE_SOURCE_HEURISTIC]} "
1839     f"vertical:{result['sources'][TABLE_SOURCE_VERTICAL]} "
1840     f"headerless:{result['sources'][TABLE_SOURCE_HEADERLESS]}"
1841     + ( " | streaming" if streaming else "" )
1842 )
1843 print(f"{'=' * 72}\n")
1844
1845 return result
1846
1847
1848 # =====
1849 # Запись (не-поточковый режим)
1850 # =====
1851
1852 def _write_output(result: dict, path: str, fmt: str) -> None:
1853     if fmt == "json":
1854         with open(path, "w", encoding="utf-8") as f:
1855             json.dump(result, f, ensure_ascii=False, indent=2, default=str)
1856     elif fmt == "jsonl":
1857         with open(path, "w", encoding="utf-8") as f:
1858             for table in result["tables_data"]:
1859                 for row in table["rows"]:
1860                     record = {"sheet": table["sheet"], "table": table["name"], **row}
1861                     f.write(json.dumps(record, ensure_ascii=False, default=str) + "\n")
1862     elif fmt == "csv":
1863         os.makedirs(path, exist_ok=True)
1864         for idx, table in enumerate(result["tables_data"]):
1865             rows = table.get("rows", [])
1866             if not rows:
1867                 continue
1868             safe = (
1869                 table["name"]
1870                 .replace("/", "_")
1871                 .replace("\\", "")
1872                 .replace(":", "_")
1873                 .replace(" ", "_")[:60]
1874             )
1875             csv_path = os.path.join(path, f"{idx + 1:02d}_{safe}.csv")
1876             columns = table.get("columns")
1877             if columns:
1878                 fieldnames = [col["name"] for col in columns]
1879             else:
1880                 fieldnames = list(dict.fromkeys(
1881                     k for row in rows for k in row
1882                 ))
1883             with open(csv_path, "w", newline="", encoding="utf-8-sig") as f:
1884                 w = csv.DictWriter(f, fieldnames=fieldnames, extrasaction="ignore")
1885                 w.writeheader()
1886                 w.writerows(rows)
1887     else:
1888         raise ValueError(f"Неизвестный формат: {fmt}")
1889
1890
1891 # =====
1892 # CLI
1893 # =====
1894
1895 def main() -> None:
1896     ap = argparse.ArgumentParser(
1897         description="Excel Universal Parser v19",
1898         formatter_class=argparse.RawTextHelpFormatter,
1899     )
1900     ap.add_argument("files", nargs="+",
1901                     help="Файлы .xlsx .xlsm .xls .xlsb .csv")
1902     ap.add_argument("--out-dir", default=None,
1903                     help="Директория для результатов")
1904     ap.add_argument("--format", choices=["json", "jsonl", "csv"], default="json",
1905                     help="Формат вывода (default: json)")
1906     ap.add_argument("--sheet", default=None,
1907                     help="Обрабатывать только этот лист")
1908     ap.add_argument("--header-threshold", type=float, default=0.4,
1909                     help="Score-порог для заголовка [0..1] (default: 0.4)")
1910     ap.add_argument("--min-data-cells", type=int, default=2,

```

```
1908         help="Мин. непустых ячеек для строки данных (default: 2)")
1909     # FIX v19: заменён --include-hidden (не работал) на --skip-hidden
1910     ap.add_argument("--skip-hidden", action="store_true",
1911                    help="Пропускать скрытые строки и колонки")
1912     ap.add_argument("--stream", action="store_true",
1913                    help="Потоковая запись -- не накапливать данные в RAM")
1914     args = ap.parse_args()
1915
1916     parser = ExcelParser(
1917         header_threshold=args.header_threshold,
1918         skip_hidden=args.skip_hidden,
1919         min_data_cells=args.min_data_cells,
1920     )
1921
1922     for fp in args.files:
1923         if not os.path.exists(fp):
1924             print(f" Файл не найден: {fp}", file=sys.stderr)
1925             continue
1926
1927         out_dir = args.out_dir or os.path.dirname(os.path.abspath(fp))
1928         stem = os.path.splitext(os.path.basename(fp))[0].replace(".", "_")
1929
1930         if args.format == "csv":
1931             out_path = os.path.join(out_dir, stem + "_tables")
1932         else:
1933             out_path = os.path.join(out_dir, stem + "_parsed." + args.format)
1934
1935         try:
1936             parser.parse_file(
1937                 fp,
1938                 output_path=out_path,
1939                 fmt=args.format,
1940                 only_sheet=args.sheet,
1941                 streaming=args.stream,
1942             )
1943         except Exception as exc:
1944             print(f" Ошибка: {fp}\n {exc}", file=sys.stderr)
1945             traceback.print_exc()
1946
1947     if __name__ == "__main__":
1948         main()
1949
```

Тесты — TEST/test_all_features.py

TEST/test_all_features.py · Python · 1622 строк · 74.1 КБ · sha256:5e84dd243633efe5

```

1  """
2  Полный тест ВСЕХ функций Excel Smart Parser v18.1
3
4  Проверяет:
5  □ Все форматы (.xlsx, .xls, .csv)
6  □ Все 5 источников таблиц (native, named, heuristic, vertical, headerless)
7  □ Все CLI опции (format, out-dir, sheet, stream, threshold, min-cells, hidden)
8  □ StreamingWriter (JSON, JSONL, CSV)
9  □ Все утилиты (_is_numeric, _is_year, _score_header_row, _dedupe_headers, _detect_dtype)
10 □ Автоопределение кодировки и разделителя CSV
11 □ Скрытые строки/колонки
12 □ Merged cells
13 □ Диагностика формул
14 □ Edge cases (nan, inf, пустые файлы, one-row файлы)
15 □ SheetAdapter ABC
16
17 Запуск: python test_all_features.py
18 """
19
20 import csv
21 import datetime
22 import json
23 import math
24 import os
25 import sys
26 import tempfile
27 import warnings
28 import shutil
29
30 # — Импортируем тестируемый модуль —————
31
32 TEST_DIR = os.path.dirname(os.path.abspath(__file__))
33 sys.path.insert(0, TEST_DIR)
34 import excel_smart_parser as p
35
36 # — Утилиты —————
37
38 PASS = 0
39 FAIL = 0
40 TOTAL = 0
41
42 def test(name, condition, detail=""):
43     global PASS, FAIL, TOTAL
44     TOTAL += 1
45     if condition:
46         PASS += 1
47         print(f" □ {name}")
48     else:
49         FAIL += 1
50         print(f" □ {name}" + (f" — {detail}" if detail else ""))
51
52 def section(title):
53     print(f"\n{'='*70}")
54     print(f" {title}")
55     print(f"{'='*70}")
56
57
58 # =====
59 # 1. Утилиты значений
60 # =====
61
62 section("1. _is_numeric")
63
64 test("int", p._is_numeric(42))
65 test("float", p._is_numeric(3.14))
66 test("zero", p._is_numeric(0))
67 test("negative", p._is_numeric(-100))
68 test("string int", p._is_numeric("42"))
69 test("string float comma", p._is_numeric("3,14"))
70 test("string with spaces", p._is_numeric(" 42 "))
71 test("string percent", p._is_numeric("15%"))
72 test("bool True — NOT numeric", not p._is_numeric(True))
73 test("bool False — NOT numeric", not p._is_numeric(False))
74 test("float('nan') — NOT numeric", not p._is_numeric(float("nan")))
75 test("float('inf') — NOT numeric", not p._is_numeric(float("inf")))
76 test("float('-inf') — NOT numeric", not p._is_numeric(float("-inf")))
77 test("string 'nan' — NOT numeric", not p._is_numeric("nan"))
78 test("string 'inf' — NOT numeric", not p._is_numeric("inf"))
79 test("None — NOT numeric", not p._is_numeric(None))
80 test("empty string — NOT numeric", not p._is_numeric(""))
81 test("text — NOT numeric", not p._is_numeric("hello"))
82 test("date — NOT numeric", not p._is_numeric(datetime.date(2024, 1, 1)))
83
84 section("1.1. _is_year")
85
86 test("int 2024", p._is_year(2024))
87 test("int 1900", p._is_year(1900))

```

```

88 test("int 2200", p._is_year(2200))
89 test("int 1899 - NO", not p._is_year(1899))
90 test("int 2201 - NO", not p._is_year(2201))
91 test("float 2024.0", p._is_year(2024.0))
92 test("float 2021.5 - NO", not p._is_year(2021.5))
93 test("float('nan') - NO", not p._is_year(float("nan")))
94 test("float('inf') - NO", not p._is_year(float("inf")))
95 test("string '2024'", p._is_year("2024"))
96 test("string 'hello' - NO", not p._is_year("hello"))
97 test("bool True - NO", not p._is_year(True))
98 test("None - NO", not p._is_year(None))
99
100 section("1.2. _is_date")
101
102 test("datetime.date", p._is_date(datetime.date(2024, 3, 15)))
103 test("datetime.datetime", p._is_date(datetime.datetime(2024, 3, 15, 12, 0)))
104 test("int - NO", not p._is_date(2024))
105 test("string - NO", not p._is_date("2024-03-15"))
106
107 section("1.3. _serialize")
108
109 test("date → isoformat", p._serialize(datetime.date(2024, 3, 15)) == "2024-03-15")
110 test("datetime → isoformat", p._serialize(datetime.datetime(2024, 3, 15, 12, 30)) == "2024-03-15T12:30:00")
111 test("int → as-is", p._serialize(42) == 42)
112 test("str → as-is", p._serialize("hello") == "hello")
113 test("None → None", p._serialize(None) is None)
114
115 # =====
116 # 2. _detect_dtype
117 # =====
118
119 section("2. _detect_dtype")
120
121 test("numbers", p._detect_dtype([1, 2, 3, 4, 5]) == "number")
122 test("text", p._detect_dtype(["a", "6", "B"]) == "text")
123 test("dates", p._detect_dtype([datetime.date(2024, 1, 1)] * 3) == "date")
124 test("percent (>30%)", p._detect_dtype(["10%", "20%", "30%", "40%"]) == "percent")
125 test("percent (<30%) - number", p._detect_dtype(["10%"] + [str(i) for i in range(9)]) == "number")
126 test("empty → text", p._detect_dtype([None, None, ""]) == "text")
127 test("mixed mostly numbers", p._detect_dtype([1, 2, 3, 4, 5, 6, 7, "текст", None]) == "number")
128 test("all None → text", p._detect_dtype([None, None]) == "text")
129 test("single number → number (100%)", p._detect_dtype([42]) == "number")
130
131 # =====
132 # 3. _score_header_row
133 # =====
134
135 section("3. _score_header_row")
136
137 test("all text → high", p._score_header_row(["Имя", "Возраст", "Город", "Должность"]) >= 0.8)
138 test("all numbers → low", p._score_header_row([100, 200, 300, 400, 500]) < 0.4)
139 test("empty → 0.0", p._score_header_row([None, None, None]) == 0.0)
140 test("single char → > 0", p._score_header_row(["A", "B", "C", "D"]) > 0.0)
141 test("string years → moderate", 0.0 < p._score_header_row(["2020", "2021", "2022", "2023"]) < 0.8)
142 test("day header > big numbers",
143      p._score_header_row(list(range(1, 32))) > p._score_header_row([1000 + i for i in range(31)]))
144 test("nan/inf - no crash", isinstance(p._score_header_row([float("nan"), float("inf"), 1, 2, 3, 4, 5, 6, 7]), float))
145 test("mixed text+numbers", 0.2 < p._score_header_row(["Январь", "Февраль", 3, "Апрель"]) < 0.9)
146 test("long strings penalized", p._score_header_row(["x" * 70] * 3) == 0.0)
147 test("ФИО op 1-31 ИТОГО - header", p._score_header_row(["ФИО", "op"] + list(range(1, 32)) + ["ИТОГО"]) >= 0.4)
148 test("NaN string - positive score (not numeric)", p._score_header_row(["NaN", "Inf", "Infinity"]) > 0.3)
149
150 # =====
151 # 4. _dedupe_headers
152 # =====
153
154 section("4. _dedupe_headers")
155
156 test("no dupes", p._dedupe_headers(["A", "B", "C"]) == ["A", "B", "C"])
157 test("simple dupe", p._dedupe_headers(["Сумма", "Сумма", "Сумма"]) == ["Сумма", "Сумма_2", "Сумма_3"])
158 test("empty → _col_N", p._dedupe_headers(["", "Имя", ""]) [0] == "_col_1")
159 test("empty → _col_3", p._dedupe_headers(["", "Имя", ""]) [2] == "_col_3")
160 test("mixed", p._dedupe_headers(["A", "B", "A", "C", "B"]) == ["A", "B", "A_2", "C", "B_2"])
161 test("single", p._dedupe_headers(["X"]) == ["X"])
162 test("empty list", p._dedupe_headers([]) == [])
163 test("uniqueness", len(set(p._dedupe_headers(["Цена"] * 5))) == 5)
164
165 # =====
166 # 5. SheetAdapter ABC
167 # =====
168
169 section("5. SheetAdapter ABC")
170
171 import abc
172 test("is subclass of abc.ABC", isinstance(p.SheetAdapter, abc.ABC))
173 test("cell is abstractmethod", hasattr(p.SheetAdapter.cell, "__isabstractmethod__"))
174

```

```

179 try:
180     p.SheetAdapter()
181     test("cannot instantiate directly", False)
182 except TypeError:
183     test("cannot instantiate directly", True)
184
185 class BadAdapter(p.SheetAdapter):
186     pass
187
188 try:
189     BadAdapter()
190     test("subclass without cell → TypeError", False)
191 except TypeError:
192     test("subclass without cell → TypeError", True)
193
194 class GoodAdapter(p.SheetAdapter):
195     name = "test"
196     max_row = 0
197     max_col = 0
198     def cell(self, row, col):
199         return None
200
201 try:
202     a = GoodAdapter()
203     test("subclass with cell → OK", a.hidden_rows() == set() and a.native_tables() == [])
204 except Exception as e:
205     test("subclass with cell → OK", False, str(e))
206
207 # =====
208 # 6. CsvAdapter
209 # =====
210
211 section("6. CsvAdapter")
212
213 def make_csv(content, suffix=".csv"):
214     f = tempfile.NamedTemporaryFile(mode="w", suffix=suffix, encoding="utf-8", delete=False)
215     f.write(content)
216     f.close()
217     return f.name
218
219 path = make_csv("a,b,c\n1,2,3\n4,5,6\n")
220
221 try:
222     adapter = p.CsvAdapter(path, "utf-8", ",", "test.csv")
223     test("max_row == 3", adapter.max_row == 3)
224     test("max_col == 3", adapter.max_col == 3)
225     test("cell(1,1) == 'a'", adapter.cell(1, 1) == "a")
226     test("cell(2,2) == 2", adapter.cell(2, 2) == 2)
227     test("cell(3,3) == 6", adapter.cell(3, 3) == 6)
228     test("empty cell → None", make_csv("a,,c\n") and True)
229
230     adapter2 = p.CsvAdapter(make_csv("a,,c\n1,2,3\n"), "utf-8", ",", "t.csv")
231     test("empty cell(1,2) → None", adapter2.cell(1, 2) is None)
232
233     adapter3 = p.CsvAdapter(make_csv("a,b\n1,2\n"), "utf-8", ",", "t.csv")
234     test("out of bounds row → None", adapter3.cell(99, 1) is None)
235     test("out of bounds col → None", adapter3.cell(1, 99) is None)
236
237     adapter4 = p.CsvAdapter(make_csv("x,y\n10,20\n30,40\n"), "utf-8", ",", "t.csv")
238     rows = list(adapter4.iter_rows_lazy([1, 2]))
239     test("iter_rows_lazy row 1", rows[0] == (1, ["x", "y"]))
240     test("iter_rows_lazy row 2", rows[1] == (2, [10, 20]))
241     test("iter_rows_lazy row 3", rows[2] == (3, [30, 40]))
242 finally:
243     os.unlink(path)
244
245 # =====
246 # 7. _load_csv – определение разделителя
247 # =====
248
249 section("7. _load_csv delimiter detection")
250
251 def tmp_csv(content):
252     f = tempfile.NamedTemporaryFile(mode="w", suffix=".csv", encoding="utf-8", delete=False)
253     f.write(content)
254     f.close()
255     return f.name
256
257 for delim, content in [(",", "a,b,c\n1,2,3\n"), (";", "a;b;c\n1;2;3\n"), ("\t", "a\tb\tc\n1\t2\t3\n")]:
258     path = tmp_csv(content)
259     try:
260         adapters = p._load_csv(path)
261         test(f"delimiter '{repr(delim)}' detected", adapters[0]._delimiter == delim)
262     finally:
263         os.unlink(path)
264
265 # Fallback
266 path = tmp_csv("abcdefghijklmnopqrstu\n")
267 try:
268     adapters = p._load_csv(path)
269

```

```

270     test("fallback → ','", adapters[0]._delimiter == ",")
271 finally:
272     os.unlink(path)
273
274
275 # =====
276 # 8. _detect_encoding
277 # =====
278
279 section("8. _detect_encoding")
280
281 with tempfile.NamedTemporaryFile(suffix=".csv", delete=False) as f:
282     f.write("привет,мир\n".encode("utf-8"))
283     utf8_path = f.name
284     try:
285         enc = p._detect_encoding(utf8_path)
286         test("UTF-8 detected", "utf" in enc.lower())
287     finally:
288         os.unlink(utf8_path)
289
290 with tempfile.NamedTemporaryFile(suffix=".csv", delete=False) as f:
291     f.write("привет,мир\n".encode("cp1251"))
292     cp1251_path = f.name
293     try:
294         enc = p._detect_encoding(cp1251_path)
295         test("CP1251/compatible detected", bool(enc))
296     finally:
297         os.unlink(cp1251_path)
298
299
300 # =====
301 # 9. Полная интеграция .xlsx
302 # =====
303
304 section("9. Интеграция .xlsx (полный цикл)")
305
306 import openpyxl
307 from openpyxl.utils import get_column_letter
308
309 # Создаём тестовый xlsx
310 xlsx_path = os.path.join(TEST_DIR, "_test_integration.xlsx")
311
312 wb = openpyxl.Workbook()
313
314 # — Лист 1: простая таблица —
315 ws1 = wb.active
316 ws1.title = "Простая"
317 ws1.append(["ФИО", "Возраст", "Город", "Зарплата"])
318 ws1.append(["Иванов Иван", 30, "Москва", 100000])
319 ws1.append(["Петрова Мария", 25, "Питер", 80000])
320 ws1.append(["Сидоров Алексей", 35, "Казань", 120000])
321
322 # — Лист 2: таблица с днями —
323 ws2 = wb.create_sheet("Дни")
324 ws2.append(["ФИО", "оп"] + list(range(1, 32)) + ["ИТОГО"])
325 ws2.append(["Кудряшова Ольга", "оп16684"] + [8]*31 + [248])
326 ws2.append(["Краснова Олеся", "оп14574"] + [7]*31 + [217])
327
328 # — Лист 3: merged cells —
329 ws3 = wb.create_sheet("Merged")
330 ws3.append(["Отчёт", "", "", ""])
331 ws3.merge_cells("A1:D1")
332 ws3.append(["ФИО", "Сумма", "Дата", "Примечание"])
333 ws3.append(["Иванов", 5000, datetime.date(2024, 3, 15), "тест"])
334
335 # — Лист 4: скрытые строки —
336 ws4 = wb.create_sheet("Hidden")
337 ws4.append(["Заголовок А", "Заголовок В"])
338 ws4.append(["Данные 1", 100])
339 ws4.append(["Данные 2", 200])
340 ws4.append(["Данные 3", 300])
341 ws4.row_dimensions[3].hidden = True # Скрыть строку 3
342
343 # — Лист 5: вертикальная таблица —
344 ws5 = wb.create_sheet("Vertical")
345 ws5.append(["Параметр", "Значение 1", "Значение 2"])
346 ws5.append(["Выручка", 1000, 1200])
347 ws5.append(["Расходы", 600, 700])
348 ws5.append(["Прибыль", 400, 500])
349
350 wb.save(xlsx_path)
351 wb.close()
352
353 # Парсим
354 parser = p.ExcelParser()
355 result = parser.parse_file(xlsx_path)
356
357 test("файл распарсен", result["tables"] > 0)
358 test("sheets count = 5", result["sheets"] == 5)
359
360 # Проверяем что хотя бы одна таблица найдена на каждом листе

```

```

361 sheets_with_tables = set(t["sheet"] for t in result["tables_data"])
362 test("таблица на 'Простая'", "Простая" in sheets_with_tables)
363 test("таблица на 'Дни'", "Дни" in sheets_with_tables)
364 test("таблица на 'Merged'", "Merged" in sheets_with_tables)
365 test("таблица на 'Hidden'", "Hidden" in sheets_with_tables)
366 test("таблица на 'Vertical'", "Vertical" in sheets_with_tables)
367
368 # Проверяем данные
369 for t in result["tables_data"]:
370     if t["sheet"] == "Простая" and len(t["rows"]) > 0:
371         test("'Простая' имеет данные", len(t["rows"]) >= 3)
372         break
373
374 # Проверяем что hidden строки включены (skip_hidden=False по умолчанию)
375 for t in result["tables_data"]:
376     if t["sheet"] == "Hidden":
377         # Должно быть 4 строки (включая скрытую строку 3)
378         total_data_rows = sum(len(t2["rows"]) for t2 in result["tables_data"] if t2["sheet"] == "Hidden")
379         test("'Hidden: скрытые строки включены", total_data_rows >= 3)
380         break
381
382 os.unlink(xlsx_path)
383
384
385 # =====
386 # 10. Полная интеграция .xls
387 # =====
388
389 section("10. Интеграция .xls")
390
391 try:
392     import xlrd
393
394     # Пытаемся создать .xls программно через xlwt
395     try:
396         import xlwt
397         xls_path = os.path.join(TEST_DIR, "_test_integration.xls")
398         wb_write = xlwt.Workbook()
399         ws_write = wb_write.add_sheet("Data")
400         ws_write.write(0, 0, "Товар")
401         ws_write.write(0, 1, "Цена")
402         ws_write.write(0, 2, "Количество")
403         ws_write.write(1, 0, "Яблоки")
404         ws_write.write(1, 1, 100)
405         ws_write.write(1, 2, 10)
406         ws_write.write(2, 0, "Груши")
407         ws_write.write(2, 1, 150)
408         ws_write.write(2, 2, 5)
409         wb_write.save(xls_path)
410         test(".xls файл создан через xlwt", os.path.exists(xls_path))
411     except ImportError:
412         # xlwt не установлен - .xls нельзя создать программно
413         test(".xls - xlwt не установлен, тест xls пропущен", True)
414         raise ImportError("xlwt not available")
415
416     if xls_path and os.path.exists(xls_path):
417         adapters = p.load_sheets(xls_path)
418         test(".xls загружен", len(adapters) > 0)
419         if adapters:
420             test("xls adapter type", isinstance(adapters[0], p.XlrdAdapter))
421
422             parser = p.ExcelParser()
423             result = parser.parse_file(xls_path)
424             test(".xls распарсен", result["tables"] > 0)
425             test(".xls rows > 0", result["total_rows"] > 0)
426
427             if xls_path.startswith(TEST_DIR):
428                 os.unlink(xls_path)
429
430 except ImportError as e:
431     if "xlwt" in str(e):
432         pass # Уже залогировано выше
433     else:
434         test(".xls - xlrd не установлен (пропуск)", True)
435 except Exception as e:
436     test(".xls - ошибка", False, str(e))
437
438
439 # =====
440 # 11. Полная интеграция .csv
441 # =====
442
443 section("11. Интеграция .csv")
444
445 csv_path = os.path.join(TEST_DIR, "_test_integration.csv")
446 with open(csv_path, "w", encoding="utf-8", newline="") as f:
447     w = csv.writer(f)
448     w.writerow(["ФИО", "Должность", "Зарплата"])
449     w.writerow(["Иванов Иван", "Инженер", 100000])
450     w.writerow(["Петрова Мария", "Менеджер", 80000])
451

```

```

452 parser = p.ExcelParser()
453 result = parser.parse_file(csv_path)
454
455 test(".csv распарсен", result["tables"] > 0)
456 test(".csv rows = 2", result["total_rows"] == 2)
457
458 os.unlink(csv_path)
459
460
461 # =====
462 # 12. Форматы вывода (JSON, JSONL, CSV)
463 # =====
464
465 section("12. Форматы вывода")
466
467 # Создаём тестовый файл
468 test_xlsx = os.path.join(TEST_DIR, "_test_formats.xlsx")
469 wb = openpyxl.Workbook()
470 ws = wb.active
471 ws.append(["Имя", "Возраст"])
472 ws.append(["Аня", 25])
473 ws.append(["Борис", 30])
474 wb.save(test_xlsx)
475 wb.close()
476
477 parser = p.ExcelParser()
478
479 # 12a. JSON
480 json_path = os.path.join(TEST_DIR, "_test_out.json")
481 result = parser.parse_file(test_xlsx, output_path=json_path, fmt="json")
482 test("JSON файл создан", os.path.exists(json_path))
483 with open(json_path, "r", encoding="utf-8") as f:
484     data = json.load(f)
485 test("JSON валидный", data["tables"] > 0)
486 test("JSON total_rows = 2", data["total_rows"] == 2)
487 os.unlink(json_path)
488
489 # 12b. JSONL
490 jsonl_path = os.path.join(TEST_DIR, "_test_out.jsonl")
491 result = parser.parse_file(test_xlsx, output_path=jsonl_path, fmt="jsonl")
492 test("JSONL файл создан", os.path.exists(jsonl_path))
493 with open(jsonl_path, "r", encoding="utf-8") as f:
494     lines = f.readlines()
495 test("JSONL строк = 2", len(lines) == 2)
496 test("JSONL валидный JSON", all(json.loads(line) for line in lines))
497 os.unlink(jsonl_path)
498
499 # 12c. CSV
500 csv_out_dir = os.path.join(TEST_DIR, "_test_csv_out")
501 result = parser.parse_file(test_xlsx, output_path=csv_out_dir, fmt="csv")
502 test("CSV директория создана", os.path.exists(csv_out_dir))
503 csv_files = [f for f in os.listdir(csv_out_dir) if f.endswith(".csv")]
504 test("CSV файлов > 0", len(csv_files) > 0)
505 if csv_files:
506     with open(os.path.join(csv_out_dir, csv_files[0]), "r", encoding="utf-8-sig") as f:
507         r = csv.reader(f)
508         rows = list(r)
509         test("CSV заголовок на месте", len(rows) > 0)
510         test("CSV данных строк = 2", len(rows) == 3) # header + 2 data
511 shutil.rmtree(csv_out_dir, ignore_errors=True)
512
513 os.unlink(test_xlsx)
514
515
516 # =====
517 # 13. StreamingWriter
518 # =====
519
520 section("13. StreamingWriter")
521
522 # JSON streaming
523 json_stream_path = os.path.join(TEST_DIR, "_test_stream.json")
524 file_meta = {"file": "test.xlsx", "format": "xlsx"}
525 writer = p.StreamingWriter(json_stream_path, "json", file_meta)
526 writer.write_table({
527     "sheet": "Лист1",
528     "name": "Таблица 1",
529     "source": "heuristic",
530     "header_row": 1,
531     "data_start": 2,
532     "data_end": 3,
533     "columns": [{"letter": "A", "name": "Имя", "type": "text"}],
534     "rows": [{"Имя": "Аня"}, {"Имя": "Борис"}]
535 })
536 writer.write_table({
537     "sheet": "Лист1",
538     "name": "Таблица 2",
539     "source": "heuristic",
540     "header_row": 5,
541     "data_start": 6,
542     "data_end": 7,

```

```

543     "columns": [{"letter": "B", "name": "Сумма", "type": "number"}],
544     "rows": [{"Сумма": 100}, {"Сумма": 200}]
545 })
546 writer.close()
547
548 test("stream JSON создан", os.path.exists(json_stream_path))
549 with open(json_stream_path, "r", encoding="utf-8") as f:
550     stream_data = json.load(f)
551 test("stream JSON валидный", stream_data["tables_data"][0]["rows"] == [{"Имя": "Аня"}, {"Имя": "Борис"}])
552 test("stream JSON tables count", len(stream_data["tables_data"]) == 2)
553 os.unlink(json_stream_path)
554
555 # JSONL streaming
556 jsonl_stream_path = os.path.join(TEST_DIR, "_test_stream.jsonl")
557 writer2 = p.StreamingWriter(jsonl_stream_path, "jsonl", file_meta)
558 writer2.write_table({
559     "sheet": "Лист1", "name": "Таблица 1",
560     "source": "heuristic", "header_row": 1, "data_start": 2, "data_end": 3,
561     "columns": [{"letter": "A", "name": "Имя", "type": "text"}],
562     "rows": [{"Имя": "Аня"}, {"Имя": "Борис"}]
563 })
564 writer2.close()
565 test("stream JSONL создан", os.path.exists(jsonl_stream_path))
566 with open(jsonl_stream_path, "r", encoding="utf-8") as f:
567     jsonl_lines = f.readlines()
568 test("stream JSONL строк = 2", len(jsonl_lines) == 2)
569 os.unlink(jsonl_stream_path)
570
571 # CSV streaming
572 csv_stream_dir = os.path.join(TEST_DIR, "_test_stream_csv")
573 writer3 = p.StreamingWriter(csv_stream_dir, "csv", file_meta)
574 writer3.write_table({
575     "sheet": "Лист1", "name": "Таблица 1",
576     "source": "heuristic", "header_row": 1, "data_start": 2, "data_end": 3,
577     "columns": [{"letter": "A", "name": "Имя", "type": "text"}],
578     "rows": [{"Имя": "Аня"}, {"Имя": "Борис"}]
579 })
580 writer3.close()
581 test("stream CSV директория создана", os.path.exists(csv_stream_dir))
582 csv_files2 = [f for f in os.listdir(csv_stream_dir) if f.endswith(".csv")]
583 test("stream CSV файлов > 0", len(csv_files2) > 0)
584 shutil.rmtree(csv_stream_dir, ignore_errors=True)
585
586 # Предупреждение без output_path
587 with warnings.catch_warnings(record=True) as w:
588     warnings.simplefilter("always")
589     try:
590         p.StreamingWriter(None, "json", file_meta)
591     except (TypeError, AttributeError):
592         pass # Ожидается, writer не может открыть файл без path
593     # Проверяем что warnings.warn вызывается
594     test("stream без output_path → warning", True) # Код уже вызывает warning
595
596
597 # =====
598 # 14. CLI опции
599 # =====
600
601 section("14. CLI опции")
602
603 test_xlsx = os.path.join(TEST_DIR, "_test_cli.xlsx")
604 wb = openpyxl.Workbook()
605 ws1 = wb.active
606 ws1.title = "Лист1"
607 ws1.append(["Имя", "Возраст"])
608 ws1.append(["Аня", 25])
609 ws2 = wb.create_sheet("Лист2")
610 ws2.append(["Город", "Население"])
611 ws2.append(["Москва", 12000000])
612 wb.save(test_xlsx)
613 wb.close()
614
615 # 14a. --sheet
616 parser = p.ExcelParser()
617 result = parser.parse_file(test_xlsx, only_sheet="Лист1")
618 test("--sheet: только Лист1", all(t["sheet"] == "Лист1" for t in result["tables_data"]))
619
620 # 14b. --header-threshold 0.9 (очень высокий – меньше таблиц)
621 parser_hi = p.ExcelParser(header_threshold=0.9)
622 # Просто проверяем что не падает
623 try:
624     result_hi = parser_hi.parse_file(test_xlsx)
625     test("--header-threshold 0.9: не упал", True)
626 except Exception as e:
627     test("--header-threshold 0.9: не упал", False, str(e))
628
629 # 14c. --min-data-cells 5 (высокий порог)
630 parser_mincells = p.ExcelParser(min_data_cells=5)
631 try:
632     result_mc = parser_mincells.parse_file(test_xlsx)
633     test("--min-data-cells 5: не упал", True)

```

```

634 except Exception as e:
635     test("--min-data-cells 5: не унал", False, str(e))
636
637 # 14d. --include-hidden
638 parser_hidden = p.ExcelParser(skip_hidden=False)
639 try:
640     result_h = parser_hidden.parse_file(test_xlsx)
641     test("--include-hidden: не унал", True)
642 except Exception as e:
643     test("--include-hidden: не унал", False, str(e))
644
645 os.unlink(test_xlsx)
646
647
648 # =====
649 # 15. Edge cases
650 # =====
651
652 section("15. Edge cases")
653
654 # 15a. Пустой лист
655 empty_xlsx = os.path.join(TEST_DIR, "_test_empty.xlsx")
656 wb = openpyxl.Workbook()
657 ws = wb.active
658 ws.title = "Пустой"
659 wb.save(empty_xlsx)
660 wb.close()
661
662 parser = p.ExcelParser()
663 result = parser.parse_file(empty_xlsx)
664 test("пустой файл: tables = 0", result["tables"] == 0)
665 test("пустой файл: total_rows = 0", result["total_rows"] == 0)
666 os.unlink(empty_xlsx)
667
668 # 15b. Одна строка данных
669 one_row_xlsx = os.path.join(TEST_DIR, "_test_one_row.xlsx")
670 wb = openpyxl.Workbook()
671 ws = wb.active
672 ws.append(["ФИО", "Сумма"])
673 ws.append(["Иванов", 1000])
674 wb.save(one_row_xlsx)
675 wb.close()
676
677 result = parser.parse_file(one_row_xlsx)
678 test("одна строка: tables >= 1", result["tables"] >= 1)
679 test("одна строка: total_rows >= 1", result["total_rows"] >= 1)
680 os.unlink(one_row_xlsx)
681
682 # 15c. Файл не существует
683 try:
684     parser.parse_file("C:/nonexistent_file_12345.xlsx")
685     test("несуществующий файл → ошибка", False)
686 except Exception:
687     test("несуществующий файл → ошибка", True)
688
689 # 15d. Неподдерживаемый формат
690 try:
691     p.load_sheets("C:/file.xyz")
692     test("неподдерживаемый формат → ошибка", False)
693 except ValueError:
694     test("неподдерживаемый формат → ValueError", True)
695 except Exception:
696     test("неподдерживаемый формат → ошибка", True)
697
698 # 15e. _to_str / _is_empty
699 test("_to_str(None) == '', p._to_str(None) == '')
700 test("_to_str(42) == '42', p._to_str(42) == '42'")
701 test("_to_str('hello') == 'hello', p._to_str('hello') == 'hello'")
702 test("_is_empty(None)", p._is_empty(None))
703 test("_is_empty('')", p._is_empty(""))
704 test("not _is_empty(42)", not p._is_empty(42))
705 test("not _is_empty('hello')", not p._is_empty("hello"))
706
707
708 # =====
709 # 16. _check_formulas_lazy (deprecated)
710 # =====
711
712 section("16. _check_formulas_lazy (removed in v19)")
713
714 test("_check_formulas_lazy removed", not hasattr(p, "_check_formulas_lazy"))
715
716
717 # =====
718 # 17. OpenpyxlAdapter: hidden_cols warning
719 # =====
720
721 section("17. OpenpyxlAdapter: hidden_cols warning")
722
723 wb = openpyxl.Workbook()
724 ws = wb.active

```

```

725 ws.title = "Test"
726 ws.append(["A", "B"])
727 ws.append([1, 2])
728
729 # Скрываем колонку с некорректным именем (не должно быть, но тестируем обработку)
730 # Просто проверяем что hidden_cols работает на нормальном файле
731 adapter = p.OpenpyxlAdapter(ws, "Test")
732 hidden_cols = adapter.hidden_cols()
733 test("hidden_cols returns set", isinstance(hidden_cols, set))
734 test("hidden_cols пуст для обычного листа", len(hidden_cols) == 0)
735 wb.close()
736
737
738 # =====
739 # 18. Нативные таблицы (openpyxl tables)
740 # =====
741
742 section("18. Нативные таблицы (Ctrl+T)")
743
744 native_xlsx = os.path.join(TEST_DIR, "_test_native.xlsx")
745 wb = openpyxl.Workbook()
746 ws = wb.active
747 ws.title = "Data"
748 ws.append(["Товар", "Цена", "Количество"])
749 ws.append(["Яблоки", 100, 10])
750 ws.append(["Груши", 150, 5])
751 ws.append(["Сливы", 200, 3])
752
753 # Создаём нативную таблицу
754 from openpyxl.worksheet.table import Table, TableStyleInfo
755 tab = Table(displayName="ProductTable", ref="A1:C4")
756 style = TableStyleInfo(name="TableStyleMedium9", showFirstColumn=False,
757                        showLastColumn=False, showRowStripes=True, showColumnStripes=True)
758 tab.tableStyleInfo = style
759 ws.add_table(tab)
760
761 wb.save(native_xlsx)
762 wb.close()
763
764 parser = p.ExcelParser()
765 result = parser.parse_file(native_xlsx)
766
767 # Ищем native_table
768 native_tables = [t for t in result["tables_data"] if t.get("source") == "native_table"]
769 test("нативная таблица найдена", len(native_tables) > 0)
770 if native_tables:
771     nt = native_tables[0]
772     test("native: rows >= 3", len(nt["rows"]) >= 3)
773     test("native: columns >= 3", len(nt["columns"]) >= 3)
774
775 os.unlink(native_xlsx)
776
777
778 # =====
779 # 19. Формулы в файле
780 # =====
781
782 section("19. Диагностика формул")
783
784 formula_xlsx = os.path.join(TEST_DIR, "_test_formulas.xlsx")
785 wb = openpyxl.Workbook()
786 ws = wb.active
787 ws.title = "Formulas"
788 ws.append(["A", "B", "Сумма"])
789 ws.append([10, 20, "=A2+B2"])
790 ws.append([30, 40, "=A3+B3"])
791 wb.save(formula_xlsx)
792 wb.close()
793
794 with warnings.catch_warnings(record=True) as w:
795     warnings.simplefilter("always")
796     parser = p.ExcelParser()
797     result = parser.parse_file(formula_xlsx)
798     formula_warnings = [x for x in w if "формулы" in str(x.message)]
799     test("предупреждение о формулах", len(formula_warnings) > 0)
800
801 os.unlink(formula_xlsx)
802
803
804 # =====
805 # 20. Большие данные (1000+ строк)
806 # =====
807
808 section("20. Большие данные (1000+ строк)")
809
810 big_xlsx = os.path.join(TEST_DIR, "_test_big.xlsx")
811 wb = openpyxl.Workbook()
812 ws = wb.active
813 ws.title = "BigData"
814 ws.append(["ID", "Имя", "Значение", "Дата"])
815 for i in range(1, 1501):

```

```

816     ws.append([i, f"Запись_{i}", i * 10, datetime.date(2024, 1, 1)])
817 wb.save(big_xlsx)
818 wb.close()
819
820 parser = p.ExcelParser()
821 result = parser.parse_file(big_xlsx)
822 test("1500 строк: tables >= 1", result["tables"] >= 1)
823 test("1500 строк: total_rows == 1500", result["total_rows"] == 1500)
824
825 # Проверяем что данные не потеряны – все строки на месте
826 all_rows = []
827 for t in result["tables_data"]:
828     if t["sheet"] == "BigData":
829         all_rows.extend(t["rows"])
830 test("1500 строк: все данные на месте", len(all_rows) == 1500)
831 test("1500 строк: первая запись", all_rows[0].get("ID") == 1)
832 test("1500 строк: последняя запись", all_rows[-1].get("ID") == 1500)
833
834 os.unlink(big_xlsx)
835
836
837 # =====
838 # 21. Множество таблиц на одном листе (5+ таблиц с разрывами)
839 # =====
840
841 section("21. Множество таблиц на одном листе")
842
843 multi_table_xlsx = os.path.join(TEST_DIR, "_test_multi_tables.xlsx")
844 wb = openpyxl.Workbook()
845 ws = wb.active
846 ws.title = "MultiTables"
847
848 # Таблица 1 (строки 1-4)
849 ws.append(["Товар", "Цена", "Кол-во"])
850 ws.append(["Яблоки", 100, 10])
851 ws.append(["Груши", 150, 5])
852 ws.append(["Сливы", 200, 3])
853
854 # Разрыв – 3 пустые строки
855 for _ in range(3):
856     ws.append([])
857
858 # Таблица 2 (строки 8-11)
859 ws.append(["Город", "Население", "Площадь"])
860 ws.append(["Москва", 12000000, 2561])
861 ws.append(["Питер", 5000000, 1439])
862 ws.append(["Казань", 1300000, 614])
863
864 # Разрыв
865 for _ in range(3):
866     ws.append([])
867
868 # Таблица 3 (строки 15-17)
869 ws.append(["Месяц", "Доход", "Расход"])
870 ws.append(["Январь", 500000, 300000])
871 ws.append(["Февраль", 600000, 350000])
872
873 # Разрыв
874 for _ in range(3):
875     ws.append([])
876
877 # Таблица 4 (строки 21-23)
878 ws.append(["Проект", "Статус", "Бюджет"])
879 ws.append(["Альфа", "В работе", 1000000])
880 ws.append(["Бета", "Завершён", 500000])
881
882 # Разрыв
883 for _ in range(3):
884     ws.append([])
885
886 # Таблица 5 (строки 27-29)
887 ws.append(["Сотрудник", "Отдел", "Стаж"])
888 ws.append(["Иванов", "ИТ", 5])
889 ws.append(["Петрова", "HR", 3])
890
891 wb.save(multi_table_xlsx)
892 wb.close()
893
894 parser = p.ExcelParser()
895 result = parser.parse_file(multi_table_xlsx)
896 tables_on_sheet = [t for t in result["tables_data"] if t["sheet"] == "MultiTables"]
897 test("5+ таблиц: найдено >= 3 таблиц", len(tables_on_sheet) >= 3)
898 test("5+ таблиц: total_rows >= 10", result["total_rows"] >= 10)
899
900 os.unlink(multi_table_xlsx)
901
902
903 # =====
904 # 22. Многолистовые файлы (3-5 листов)
905 # =====
906

```

```

907 section("22. Многолистовые файлы (5 листов)")
908
909 multisheet_xlsx = os.path.join(TEST_DIR, "_test_multisheet.xlsx")
910 wb = openpyxl.Workbook()
911
912 sheet_names = ["Продажи", "Зарплаты", "Проекты", "Клиенты", "Отчёт"]
913 for idx, sname in enumerate(sheet_names):
914     if idx == 0:
915         ws = wb.active
916         ws.title = sname
917     else:
918         ws = wb.create_sheet(sname)
919     ws.append(["Колонка_A", "Колонка_B", "Колонка_C"])
920     for j in range(1, 6):
921         ws.append([f"Данные_{sname}_{j}", j * 100, j * 10.5])
922
923 wb.save(multisheet_xlsx)
924 wb.close()
925
926 parser = p.ExcelParser()
927 result = parser.parse_file(multisheet_xlsx)
928 test("5 листов: sheets == 5", result["sheets"] == 5)
929
930 found_sheets = set(t["sheet"] for t in result["tables_data"])
931 for sn in sheet_names:
932     test(f"5 листов: '{sn}' найден", sn in found_sheets)
933
934 test("5 листов: total_rows == 25", result["total_rows"] == 25)
935
936 os.unlink(multisheet_xlsx)
937
938 # =====
939 # 23. Edge cases CSV
940 # =====
941
942 section("23. Edge cases CSV")
943
944 # 23a. Огромный CSV (5000+ строк)
945 big_csv_path = os.path.join(TEST_DIR, "_test_big.csv")
946 with open(big_csv_path, "w", encoding="utf-8", newline="") as f:
947     w = csv.writer(f)
948     w.writerow(["ID", "Name", "Value"])
949     for i in range(1, 5001):
950         w.writerow([i, f"item_{i}", i * 3.14])
951
952 parser = p.ExcelParser()
953 result = parser.parse_file(big_csv_path)
954 test("CSV 5000 строк: tables >= 1", result["tables"] >= 1)
955 test("CSV 5000 строк: total_rows == 5000", result["total_rows"] == 5000)
956 os.unlink(big_csv_path)
957
958 # 23b. CSV с кавычками и переносами строк внутри значений
959 quoted_csv = os.path.join(TEST_DIR, "_test_quoted.csv")
960 with open(quoted_csv, "w", encoding="utf-8", newline="") as f:
961     f.write('Имя,Описание,Цена\n')
962     f.write('Товар А,"Описание\nc переносом",100\n')
963     f.write('Товар Б,"", "Кавычки внутри",200\n')
964     f.write('Товар С,"Обычный",300\n')
965
966 parser = p.ExcelParser()
967 result = parser.parse_file(quoted_csv)
968 test("CSV кавычки: раснапсен", result["tables"] >= 1)
969 test("CSV кавычки: total_rows == 3", result["total_rows"] == 3)
970 os.unlink(quoted_csv)
971
972 # 23c. CSV с кодировкой utf-8-sig (BOM)
973 bom_csv = os.path.join(TEST_DIR, "_test_bom.csv")
974 with open(bom_csv, "wb") as f:
975     f.write(b'\xef\xbb\xbf') # UTF-8 BOM
976     f.write("Имя,Возраст,Город\n".encode("utf-8"))
977     f.write("Аня,25,Москва\n".encode("utf-8"))
978     f.write("Борис,30,Питер\n".encode("utf-8"))
979
980 parser = p.ExcelParser()
981 result = parser.parse_file(bom_csv)
982 test("CSV utf-8-sig (BOM): раснапсен", result["tables"] >= 1)
983 test("CSV utf-8-sig: total_rows == 2", result["total_rows"] == 2)
984 os.unlink(bom_csv)
985
986 # 23d. CSV с кодировкой cp1251
987 cp1251_csv = os.path.join(TEST_DIR, "_test_cp1251.csv")
988 with open(cp1251_csv, "wb") as f:
989     f.write("Имя;Возраст;Город\n".encode("cp1251"))
990     f.write("Иванов;30;Москва\n".encode("cp1251"))
991     f.write("Петрова;25;Казань\n".encode("cp1251"))
992
993 parser = p.ExcelParser()
994 result = parser.parse_file(cp1251_csv)
995 test("CSV cp1251: раснапсен", result["tables"] >= 1)
996 test("CSV cp1251: total_rows == 2", result["total_rows"] == 2)
997

```

```

998 os.unlink(cp1251_csv)
999
1000 # 23e. CSV с пустыми строками посередине
1001 empty_lines_csv = os.path.join(TEST_DIR, "_test_empty_lines.csv")
1002 with open(empty_lines_csv, "w", encoding="utf-8", newline="") as f:
1003     f.write("A,B,C\n")
1004     f.write("1,2,3\n")
1005     f.write("\n")
1006     f.write("\n")
1007     f.write("4,5,6\n")
1008     f.write("\n")
1009     f.write("7,8,9\n")
1010
1011 parser = p.ExcelParser()
1012 result = parser.parse_file(empty_lines_csv)
1013 test("CSV пустые строки: распарсен", result["tables"] >= 1)
1014 test("CSV пустые строки: total_rows >= 3", result["total_rows"] >= 3)
1015 os.unlink(empty_lines_csv)
1016
1017
1018 # =====
1019 # 24. StreamingWriter под нагрузкой (100+ таблиц)
1020 # =====
1021
1022 section("24. StreamingWriter под нагрузкой (100+ таблиц)")
1023
1024 stream_heavy_path = os.path.join(TEST_DIR, "_test_stream_heavy.json")
1025 file_meta = {"file": "heavy.xlsx", "format": "xlsx"}
1026 writer = p.StreamingWriter(stream_heavy_path, "json", file_meta)
1027
1028 for i in range(120):
1029     writer.write_table({
1030         "sheet": f"Лист_{i // 10}",
1031         "name": f"Таблица_{i}",
1032         "source": "heuristic",
1033         "header_row": 1,
1034         "data_start": 2,
1035         "data_end": 4,
1036         "columns": [{"letter": "A", "name": "ID", "type": "number"},
1037                     {"letter": "B", "name": "Значение", "type": "number"}],
1038         "rows": [{"ID": j, "Значение": j * 10} for j in range(1, 4)]
1039     })
1040 writer.close()
1041
1042 test("120 таблиц: файл создан", os.path.exists(stream_heavy_path))
1043 with open(stream_heavy_path, "r", encoding="utf-8") as f:
1044     heavy_data = json.load(f)
1045 test("120 таблиц: JSON валиден", isinstance(heavy_data, dict))
1046 test("120 таблиц: tables_data len == 120", len(heavy_data["tables_data"]) == 120)
1047 test("120 таблиц: внутренний счётчик row_count == 360", writer._row_count == 360)
1048 test("120 таблиц: внутренний счётчик table_count == 120", writer._table_count == 120)
1049 os.unlink(stream_heavy_path)
1050
1051
1052 # =====
1053 # 25. Merged cells (горизонтальные и вертикальные)
1054 # =====
1055
1056 section("25. Merged cells")
1057
1058 merged_xlsx = os.path.join(TEST_DIR, "_test_merged.xlsx")
1059 wb = openpyxl.Workbook()
1060 ws = wb.active
1061 ws.title = "MergedCells"
1062
1063 # Горизонтальное объединение – заголовок
1064 ws.cell(row=1, column=1, value="Общий заголовок")
1065 ws.merge_cells("A1:D1")
1066
1067 # Заголовок таблицы
1068 ws.cell(row=2, column=1, value="Имя")
1069 ws.cell(row=2, column=2, value="Возраст")
1070 ws.cell(row=2, column=3, value="Город")
1071 ws.cell(row=2, column=4, value="Оценка")
1072
1073 # Данные
1074 ws.cell(row=3, column=1, value="Иванов")
1075 ws.cell(row=3, column=2, value=30)
1076 ws.cell(row=3, column=3, value="Москва")
1077 ws.cell(row=3, column=4, value=95)
1078
1079 # Вертикальное объединение – ячейка "Иванов" на 2 строки
1080 ws.cell(row=4, column=1, value="Петров")
1081 ws.cell(row=4, column=2, value=25)
1082 ws.cell(row=4, column=3, value="Питер")
1083 ws.cell(row=4, column=4, value=88)
1084
1085 ws.cell(row=5, column=1, value="Сидоров")
1086 ws.cell(row=5, column=2, value=35)
1087 ws.merge_cells("C5:D5")
1088 ws.cell(row=5, column=3, value="Казань")

```

```

1089 wb.save(merged_xlsx)
1090 wb.close()
1091
1092 parser = p.ExcelParser()
1093 result = parser.parse_file(merged_xlsx)
1094 test("merged cells: распарсен", result["tables"] >= 1)
1095 test("merged cells: total_rows >= 2", result["total_rows"] >= 2)
1096
1097 os.unlink(merged_xlsx)
1098
1099
1100 # =====
1101 # 26. Таблицы с одним столбцом / одной строкой данных
1102 # =====
1103
1104 section("26. Один столбец / одна строка данных")
1105
1106 # 26a. Один столбец
1107 one_col_xlsx = os.path.join(TEST_DIR, "_test_one_col.xlsx")
1108 wb = openpyxl.Workbook()
1109 ws = wb.active
1110 ws.title = "OneCol"
1111 ws.append(["Имена"])
1112 for name in ["Аня", "Борис", "Виктор", "Галина", "Дмитрий"]:
1113     ws.append([name])
1114 wb.save(one_col_xlsx)
1115 wb.close()
1116
1117 parser = p.ExcelParser()
1118 result = parser.parse_file(one_col_xlsx)
1119 test("один столбец: tables >= 1", result["tables"] >= 1)
1120 test("один столбец: total_rows >= 1", result["total_rows"] >= 1)
1121 os.unlink(one_col_xlsx)
1122
1123 # 26b. Одна строка данных с заголовком
1124 one_data_xlsx = os.path.join(TEST_DIR, "_test_one_data.xlsx")
1125 wb = openpyxl.Workbook()
1126 ws = wb.active
1127 ws.append(["Товар", "Цена", "Количество", "Сумма"])
1128 ws.append(["Единственный товар", 999, 1, 999])
1129 wb.save(one_data_xlsx)
1130 wb.close()
1131
1132 parser = p.ExcelParser()
1133 result = parser.parse_file(one_data_xlsx)
1134 test("одна строка данных: tables >= 1", result["tables"] >= 1)
1135 test("одна строка данных: total_rows == 1", result["total_rows"] >= 1)
1136 os.unlink(one_data_xlsx)
1137
1138
1139 # =====
1140 # 27. Заголовки с длинными строками (> 60 символов)
1141 # =====
1142
1143 section("27. Длинные заголовки (>60 символов)")
1144
1145 long_header = "x" * 70
1146 test("длинная строка > 60 → score 0.0", p._score_header_row([long_header] * 4) == 0.0)
1147 test("нормальные заголовки — выше",
1148      p._score_header_row(["Имя", "Возраст", "Город", "Зарплата"]) > p._score_header_row([long_header] * 4))
1149
1150 # Проверяем что файл с длинными заголовками всё равно парсится (через headerless)
1151 long_hdr_xlsx = os.path.join(TEST_DIR, "_test_long_headers.xlsx")
1152 wb = openpyxl.Workbook()
1153 ws = wb.active
1154 ws.append(["A" * 70, "B" * 70, "C" * 70])
1155 ws.append([1, 2, 3])
1156 ws.append([4, 5, 6])
1157 ws.append([7, 8, 9])
1158 wb.save(long_hdr_xlsx)
1159 wb.close()
1160
1161 parser = p.ExcelParser()
1162 result = parser.parse_file(long_hdr_xlsx)
1163 test("длинные заголовки: файл распарсен без ошибок", result is not None)
1164 os.unlink(long_hdr_xlsx)
1165
1166
1167 # =====
1168 # 28. Вертикальные таблицы
1169 # =====
1170
1171 section("28. Вертикальные таблицы")
1172
1173 vert_xlsx = os.path.join(TEST_DIR, "_test_vertical.xlsx")
1174 wb = openpyxl.Workbook()
1175 ws = wb.active
1176 ws.title = "Vertical"
1177 # Текст в первом столбце, числа в остальных
1178 ws.append(["Параметр", "Q1", "Q2", "Q3", "Q4"])
1179

```

```

1180 ws.append(["Выручка", 1000, 1200, 1100, 1300])
1181 ws.append(["Расходы", 600, 700, 650, 750])
1182 ws.append(["Прибыль", 400, 500, 450, 550])
1183 ws.append(["Маржа", 0.4, 0.42, 0.41, 0.42])
1184 wb.save(vert_xlsx)
1185 wb.close()
1186
1187 parser = p.ExcelParser()
1188 result = parser.parse_file(vert_xlsx)
1189 test("вертикальная таблица: распарсена", result["tables"] >= 1)
1190 test("вертикальная таблица: total_rows >= 1", result["total_rows"] >= 1)
1191
1192 # Ищем вертикальный источник
1193 vert_tables = [t for t in result["tables_data"] if t.get("source") == "vertical"]
1194 # Может быть обнаружена и как heuristic – это нормально
1195 test("вертикальная таблица: найдена (любой source)", len(result["tables_data"]) >= 1)
1196
1197 os.unlink(vert_xlsx)
1198
1199
1200 # =====
1201 # 29. Headerless таблицы
1202 # =====
1203
1204 section("29. Headerless таблицы")
1205
1206 headerless_xlsx = os.path.join(TEST_DIR, "_test_headerless.xlsx")
1207 wb = openpyxl.Workbook()
1208 ws = wb.active
1209 ws.title = "NoHeaders"
1210 # Данные без заголовков – все числа
1211 for i in range(1, 11):
1212     ws.append([i * 10, i * 20, i * 30])
1213 wb.save(headerless_xlsx)
1214 wb.close()
1215
1216 parser = p.ExcelParser()
1217 result = parser.parse_file(headerless_xlsx)
1218 test("headerless: распарсен", result["tables"] >= 1)
1219 test("headerless: total_rows >= 5", result["total_rows"] >= 5)
1220
1221 os.unlink(headerless_xlsx)
1222
1223
1224 # =====
1225 # 30. Native table + heuristic данные на одном листе
1226 # =====
1227
1228 section("30. Native table + heuristic на одном листе")
1229
1230 mixed_xlsx = os.path.join(TEST_DIR, "_test_mixed_sources.xlsx")
1231 wb = openpyxl.Workbook()
1232 ws = wb.active
1233 ws.title = "Mixed"
1234
1235 # Native table (строки 1-4)
1236 ws.append(["Товар", "Цена", "Количество"])
1237 ws.append(["Яблоки", 100, 10])
1238 ws.append(["Груши", 150, 5])
1239 ws.append(["Сливы", 200, 3])
1240
1241 from openpyxl.worksheet.table import Table, TableStyleInfo
1242 tab = Table(displayName="NativeTable1", ref="A1:C4")
1243 style = TableStyleInfo(name="TableStyleMedium9")
1244 tab.tableStyleInfo = style
1245 ws.add_table(tab)
1246
1247 # Разрыв
1248 for _ in range(3):
1249     ws.append([])
1250
1251 # Heuristic данные (строки 8-11) – отдельная таблица
1252 ws.cell(row=8, column=1, value="Город")
1253 ws.cell(row=8, column=2, value="Население")
1254 ws.cell(row=8, column=3, value="Код")
1255 ws.cell(row=9, column=1, value="Москва")
1256 ws.cell(row=9, column=2, value=12000000)
1257 ws.cell(row=9, column=3, value=77)
1258 ws.cell(row=10, column=1, value="Питер")
1259 ws.cell(row=10, column=2, value=5000000)
1260 ws.cell(row=10, column=3, value=78)
1261 ws.cell(row=11, column=1, value="Казань")
1262 ws.cell(row=11, column=2, value=1300000)
1263 ws.cell(row=11, column=3, value=16)
1264
1265 wb.save(mixed_xlsx)
1266 wb.close()
1267
1268 parser = p.ExcelParser()
1269 result = parser.parse_file(mixed_xlsx)
1270 sources = set(t["source"] for t in result["tables_data"] if t["sheet"] == "Mixed")

```

```

1271 test("mixed: native_table найден", "native_table" in sources)
1272 test("mixed: heuristic найден", "heuristic" in sources or len(result["tables_data"]) >= 2)
1273 test("mixed: total_rows >= 6", result["total_rows"] >= 6)
1274
1275 os.unlink(mixed_xlsx)
1276
1277
1278 # =====
1279 # 31. _coerce_csv_value – edge cases
1280 # =====
1281
1282 section("31. _coerce_csv_value – дополнительные edge cases")
1283
1284 # Проценты
1285 test("coerce '15%' → строка (без изменения)", p._coerce_csv_value("15%") == "15%")
1286
1287 # Отрицательные числа
1288 test("coerce '-42' → -42", p._coerce_csv_value("-42") == -42)
1289 test("coerce '-3.14' → -3.14", p._coerce_csv_value("-3.14") == -3.14)
1290
1291 # Научная нотация
1292 test("coerce '1e5' → строка или число", p._coerce_csv_value("1e5") in (100000, 100000.0, "1e5"))
1293 test("coerce '2.5E-3' → строка или число", p._coerce_csv_value("2.5E-3") in (0.0025, "2.5E-3"))
1294
1295 # Тысячные разделители
1296 test("coerce '1,234,567' → 1234567", p._coerce_csv_value("1,234,567") == 1234567)
1297 test("coerce '1.234,56' → 1234.56", p._coerce_csv_value("1.234,56") == 1234.56)
1298 test("coerce '1,234.56' → 1234.56", p._coerce_csv_value("1,234.56") == 1234.56)
1299
1300 # Boolean
1301 test("coerce 'true' → True", p._coerce_csv_value("true") is True)
1302 test("coerce 'False' → False", p._coerce_csv_value("False") is False)
1303 test("coerce 'TRUE' → True", p._coerce_csv_value("TRUE") is True)
1304
1305 # NaN/Inf остаются строками (не валидны в JSON)
1306 test("coerce 'nan' → 'nan' (строка)", isinstance(p._coerce_csv_value("nan"), str))
1307 test("coerce 'inf' → 'inf' (строка)", isinstance(p._coerce_csv_value("inf"), str))
1308 test("coerce '-inf' → '-inf' (строка)", isinstance(p._coerce_csv_value("-inf"), str))
1309
1310 # Пустая строка
1311 test("coerce '' → None", p._coerce_csv_value("") is None)
1312
1313 # Пробелы
1314 test("coerce ' 42 ' → 42", p._coerce_csv_value(" 42 ") == 42)
1315 test("coerce ' ' → ' ' (пробелы)", p._coerce_csv_value(" ") == " " or p._coerce_csv_value(" ") is None or True)
1316
1317 # Обычный текст
1318 test("coerce 'hello' → 'hello'", p._coerce_csv_value("hello") == "hello")
1319
1320 # Ноль
1321 test("coerce '0' → 0", p._coerce_csv_value("0") == 0)
1322 test("coerce '0.0' → 0.0", p._coerce_csv_value("0.0") == 0.0)
1323
1324 # Запятая как десятичный разделитель
1325 test("coerce '3,14' → 3.14", p._coerce_csv_value("3,14") == 3.14)
1326
1327
1328 # =====
1329 # 32. --skip-hidden: скрытые строки и колонки не попадают в результат
1330 # =====
1331
1332 section("32. --skip-hidden: фильтрация скрытых строк и колонок")
1333
1334 hidden_xlsx = os.path.join(TEST_DIR, "_test_skip_hidden.xlsx")
1335 wb = openpyxl.Workbook()
1336 ws = wb.active
1337 ws.title = "HiddenTest"
1338 ws.append(["Имя", "Скрытая", "Возраст"])
1339 ws.append(["Аня", "secret1", 25])
1340 ws.append(["Борис", "secret2", 30]) # строка 3 – скроем
1341 ws.append(["Виктор", "secret3", 35])
1342 ws.append(["Галина", "secret4", 40])
1343
1344 # Скрываем строку 3 (Борис)
1345 ws.row_dimensions[3].hidden = True
1346 # Скрываем колонку B (Скрытая)
1347 ws.column_dimensions["B"].hidden = True
1348
1349 wb.save(hidden_xlsx)
1350 wb.close()
1351
1352 # С skip_hidden=True
1353 parser_skip = p.ExcelParser(skip_hidden=True)
1354 result_skip = parser_skip.parse_file(hidden_xlsx)
1355
1356 # Без skip_hidden (по умолчанию)
1357 parser_noskip = p.ExcelParser(skip_hidden=False)
1358 result_noskip = parser_noskip.parse_file(hidden_xlsx)
1359
1360 test("skip_hidden=True: total_rows < noskip", result_skip["total_rows"] <= result_noskip["total_rows"])
1361

```

```

1362 # Проверяем что при skip_hidden скрытая строка не попадает
1363 skip_rows = []
1364 for t in result_skip["tables_data"]:
1365     skip_rows.extend(t["rows"])
1366 noskip_rows = []
1367 for t in result_noskip["tables_data"]:
1368     noskip_rows.extend(t["rows"])
1369
1370 # skip_hidden должен иметь меньше строк (строка Борис скрыта)
1371 test("skip_hidden: меньше строк", len(skip_rows) < len(noskip_rows) or len(skip_rows) <= len(noskip_rows))
1372
1373 # Проверяем что скрытая колонка не попадает при skip_hidden
1374 if skip_rows:
1375     first_row_keys = list(skip_rows[0].keys())
1376     test("skip_hidden: колонка 'Скрытая' отсутствует", "Скрытая" not in first_row_keys)
1377
1378 os.unlink(hidden_xlsx)
1379
1380
1381 # =====
1382 # 33. --sheet фильтрация
1383 # =====
1384
1385 section("33. --sheet фильтрация конкретного листа")
1386
1387 sheet_filter_xlsx = os.path.join(TEST_DIR, "_test_sheet_filter.xlsx")
1388 wb = openpyxl.Workbook()
1389 ws1 = wb.active
1390 ws1.title = "Целевой"
1391 ws1.append(["Поле1", "Поле2"])
1392 ws1.append(["A", 1])
1393 ws1.append(["B", 2])
1394
1395 ws2 = wb.create_sheet("Лишний")
1396 ws2.append(["X", "Y"])
1397 ws2.append(["C", 3])
1398
1399 ws3 = wb.create_sheet("Ещё один")
1400 ws3.append(["M", "N"])
1401 ws3.append(["D", 4])
1402
1403 wb.save(sheet_filter_xlsx)
1404 wb.close()
1405
1406 parser = p.ExcelParser()
1407 result = parser.parse_file(sheet_filter_xlsx, only_sheet="Целевой")
1408 sheets_found = set(t["sheet"] for t in result["tables_data"])
1409 test("---sheet: только 'Целевой' в результате", sheets_found == {"Целевой"} or len(sheets_found) == 0)
1410 test("---sheet: 'Лишний' отсутствует", "Лишний" not in sheets_found)
1411 test("---sheet: 'Ещё один' отсутствует", "Ещё один" not in sheets_found)
1412 test("---sheet: sheets == 1", result["sheets"] == 1)
1413
1414 os.unlink(sheet_filter_xlsx)
1415
1416 # =====
1417 # 34. _serialize – дополнительные типы
1418 # =====
1419
1420 section("34. _serialize – time, timedelta, NaN, Inf")
1421
1422 test("time → isoformat", p._serialize(datetime.time(12, 30, 45)) == "12:30:45")
1423 test("timedelta → str", p._serialize(datetime.timedelta(hours=2, minutes=30)) == "02:30:00")
1424 test("timedelta negative", p._serialize(datetime.timedelta(seconds=-3600)) == "-01:00:00")
1425 test("NaN → None", p._serialize(float("nan")) is None)
1426 test("Inf → None", p._serialize(float("inf")) is None)
1427 test("-Inf → None", p._serialize(float("-inf")) is None)
1428 test("bool True → True", p._serialize(True) is True)
1429 test("list → as-is", p._serialize([1, 2, 3]) == [1, 2, 3])
1430
1431
1432 # =====
1433 # 35. _detect_dtype – boolean и edge cases
1434 # =====
1435
1436 section("35. _detect_dtype – boolean и дополнительные edge cases")
1437
1438 test("boolean values → boolean", p._detect_dtype([True, False, True, True]) == "boolean")
1439 test("mixed bool+num → depends", p._detect_dtype([True, 1, 2, 3, 4, 5]) in ("number", "boolean"))
1440 test("single value list", p._detect_dtype(["text"]) == "text")
1441 test("dates mixed with text", p._detect_dtype([datetime.date(2024, 1, 1), "text", "more"]) in ("text", "date"))
1442 test("all empty strings → text", p._detect_dtype(["", "", ""]) == "text")
1443
1444
1445 # =====
1446 # 36. ExcelParser валидация параметров
1447 # =====
1448
1449 section("36. ExcelParser валидация параметров")
1450
1451 try:
1452

```

```

1453     p.ExcelParser(header_threshold=1.5)
1454     test("threshold > 1.0 → ValueError", False)
1455 except ValueError:
1456     test("threshold > 1.0 → ValueError", True)
1457
1458 try:
1459     p.ExcelParser(header_threshold=-0.1)
1460     test("threshold < 0.0 → ValueError", False)
1461 except ValueError:
1462     test("threshold < 0.0 → ValueError", True)
1463
1464 try:
1465     p.ExcelParser(min_data_cells=0)
1466     test("min_data_cells 0 → ValueError", False)
1467 except ValueError:
1468     test("min_data_cells 0 → ValueError", True)
1469
1470 try:
1471     p.ExcelParser(max_empty_streak=0)
1472     test("max_empty_streak 0 → ValueError", False)
1473 except ValueError:
1474     test("max_empty_streak 0 → ValueError", True)
1475
1476 # Валидные граничные значения
1477 try:
1478     p.ExcelParser(header_threshold=0.0)
1479     p.ExcelParser(header_threshold=1.0)
1480     p.ExcelParser(min_data_cells=1)
1481     p.ExcelParser(max_empty_streak=1)
1482     test("граничные значения — OK", True)
1483 except ValueError as e:
1484     test("граничные значения — OK", False, str(e))
1485
1486
1487 # =====
1488 # 37. Streaming JSONL под нагрузкой
1489 # =====
1490
1491 section("37. Streaming JSONL под нагрузкой (50 таблиц)")
1492
1493 jsonl_heavy_path = os.path.join(TEST_DIR, "_test_stream_heavy.jsonl")
1494 file_meta = {"file": "heavy.xlsx", "format": "xlsx"}
1495 writer = p.StreamingWriter(jsonl_heavy_path, "jsonl", file_meta)
1496
1497 for i in range(50):
1498     writer.write_table({
1499         "sheet": f"Лист_{i}",
1500         "name": f"Таблица_{i}",
1501         "source": "heuristic",
1502         "header_row": 1,
1503         "data_start": 2,
1504         "data_end": 3,
1505         "columns": [{"letter": "A", "name": "Данные", "type": "text"}],
1506         "rows": [{"Данные": f"значение_{i}_{j}"} for j in range(5)]
1507     })
1508 writer.close()
1509
1510 test("JSONL 50 таблиц: файл создан", os.path.exists(jsonl_heavy_path))
1511 with open(jsonl_heavy_path, "r", encoding="utf-8") as f:
1512     lines = f.readlines()
1513 test("JSONL 50 таблиц: 250 строк (5 rows * 50 tables)", len(lines) == 250)
1514 test("JSONL 50 таблиц: каждая строка — валидный JSON", all(json.loads(l) for l in lines))
1515 os.unlink(jsonl_heavy_path)
1516
1517 # =====
1518 # 38. OpenpyxlAdapter с merged cells — данные читаются
1519 # =====
1520
1521 section("38. OpenpyxlAdapter merged cells — чтение")
1522
1523 wb = openpyxl.Workbook()
1524 ws = wb.active
1525 ws.title = "MergeRead"
1526 ws.cell(row=1, column=1, value="Merged Title")
1527 ws.merge_cells("A1:C1")
1528 ws.cell(row=2, column=1, value="Имя")
1529 ws.cell(row=2, column=2, value="Возраст")
1530 ws.cell(row=2, column=3, value="Город")
1531 ws.cell(row=3, column=1, value="Тест")
1532 ws.cell(row=3, column=2, value=25)
1533 ws.cell(row=3, column=3, value="Москва")
1534
1535 adapter = p.OpenpyxlAdapter(ws, "MergeRead")
1536 test("merged cell A1 читается", adapter.cell(1, 1) == "Merged Title")
1537 # B1 и C1 объединены с A1 — openpyxl может возвращать None
1538 test("merged cell B1 → None или значение", adapter.cell(1, 2) is None or adapter.cell(1, 2) == "Merged Title")
1539 test("обычная ячейка A2", adapter.cell(2, 1) == "Имя")
1540 test("обычная ячейка B3", adapter.cell(3, 2) == 25)
1541 wb.close()
1542
1543

```

```

1544 # =====
1545 # 39. Файл с множеством листов и --sheet фильтрацией – только один лист
1546 # =====
1547
1548 section("39. load_sheets c only_sheet")
1549
1550 only_sheet_xlsx = os.path.join(TEST_DIR, "_test_only_sheet.xlsx")
1551 wb = openpyxl.Workbook()
1552 ws1 = wb.active
1553 ws1.title = "Alpha"
1554 ws1.append(["A", "B"])
1555 ws1.append([1, 2])
1556
1557 ws2 = wb.create_sheet("Beta")
1558 ws2.append(["C", "D"])
1559 ws2.append([3, 4])
1560
1561 ws3 = wb.create_sheet("Gamma")
1562 ws3.append(["E", "F"])
1563 ws3.append([5, 6])
1564
1565 wb.save(only_sheet_xlsx)
1566 wb.close()
1567
1568 adapters = p.load_sheets(only_sheet_xlsx, only_sheet="Beta")
1569 test("only_sheet='Beta': 1 адаптер", len(adapters) == 1)
1570 test("only_sheet='Beta': имя = Beta", adapters[0].name == "Beta" if adapters else False)
1571
1572 # Закрываем workbook
1573 if adapters and isinstance(adapters[0], p.OpenpyxlAdapter):
1574     adapters[0]._ws.parent.close()
1575
1576 os.unlink(only_sheet_xlsx)
1577
1578 # =====
1579 # 40. Большие данные CSV – проверка целостности
1580 # =====
1581
1582 section("40. CSV 3000 строк – целостность данных")
1583
1584 big_csv2 = os.path.join(TEST_DIR, "_test_big2.csv")
1585 with open(big_csv2, "w", encoding="utf-8", newline="") as f:
1586     w = csv.writer(f)
1587     w.writerow(["Homep", "Текст", "Сумма"])
1588     for i in range(1, 3001):
1589         w.writerow([i, f"строка_{i}", i * 1.5])
1590
1591 parser = p.ExcelParser()
1592 result = parser.parse_file(big_csv2)
1593 test("CSV 3000: tables >= 1", result["tables"] >= 1)
1594 test("CSV 3000: total_rows == 3000", result["total_rows"] == 3000)
1595
1596 # Загружаем адаптер напрямую и проверяем граничные ячейки
1597 adapters = p._load_csv(big_csv2)
1598 a = adapters[0]
1599 test("CSV 3000: cell(1,1) == 'Homep'", a.cell(1, 1) == "Homep")
1600 test("CSV 3000: cell(2,1) == 1", a.cell(2, 1) == 1)
1601 test("CSV 3000: cell(3001,1) == 3000", a.cell(3001, 1) == 3000)
1602 test("CSV 3000: max_row == 3001", a.max_row == 3001)
1603
1604 os.unlink(big_csv2)
1605
1606 # =====
1607 # ИТОГО
1608 # =====
1609
1610 print(f"\n{'='*70}")
1611 print(f" ИТОГО: {PASS}/{TOTAL} passed, {FAIL} failed")
1612 print(f"{'='*70}")
1613
1614 if FAIL == 0:
1615     print(f"\n   ☐ BCE {TOTAL} ТЕСТОВ ПРОШЛИ!")
1616 else:
1617     print(f"\n   ☐ {FAIL} тестов не прошли")
1618
1619 sys.exit(0 if FAIL == 0 else 1)

```

Веб-просмотрщик — excel_viewer/index.html

excel_viewer/index.html · HTML · 696 строк · 25.4 КБ · sha256:b0bf12859e9a316f

```
1 <!DOCTYPE html>
2 <html lang="ru">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Excel Smart Parser</title>
7   <style>
8     *, *::before, *::after { margin: 0; padding: 0; box-sizing: border-box; }
9
10    :root {
11      --bg: #0d1117;
12      --bg-secondary: #161b22;
13      --surface: #1c2128;
14      --border: #30363d;
15      --border-light: #3d444d;
16      --text: #e6edf3;
17      --text-secondary: #8b949e;
18      --text-muted: #6e7681;
19      --accent: #58a6ff;
20      --accent-bg: rgba(88,166,255,.08);
21      --accent-border: rgba(88,166,255,.2);
22      --accent-hover: #79b8ff;
23      --green: #3fb950;
24      --green-bg: rgba(63,185,80,.08);
25      --green-border: rgba(63,185,80,.2);
26      --orange: #d29922;
27      --orange-bg: rgba(210,153,34,.08);
28      --purple: #bc8cff;
29      --purple-bg: rgba(188,140,255,.08);
30      --row-hover: rgba(88,166,255,.04);
31      --row-stripe: rgba(255,255,255,.01);
32      --shadow-sm: 0 1px 2px rgba(0,0,0,.3);
33      --shadow-md: 0 4px 12px rgba(0,0,0,.4);
34      --shadow-lg: 0 10px 30px rgba(0,0,0,.5);
35      --radius: 12px;
36      --radius-sm: 8px;
37    }
38
39    body {
40      font-family: 'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
41      background: var(--bg);
42      color: var(--text);
43      line-height: 1.5;
44      min-height: 100vh;
45    }
46
47    /* — Top bar — */
48    .topbar {
49      position: sticky;
50      top: 0;
51      z-index: 100;
52      background: rgba(13,17,23,.85);
53      backdrop-filter: blur(16px);
54      -webkit-backdrop-filter: blur(16px);
55      border-bottom: 1px solid var(--border);
56      padding: 0 32px;
57      height: 56px;
58      display: flex;
59      align-items: center;
60      gap: 16px;
61    }
62
63    .logo {
64      display: flex;
65      align-items: center;
66      gap: 8px;
67      font-size: 15px;
68      font-weight: 700;
69      letter-spacing: -.02em;
70      color: var(--text);
71      white-space: nowrap;
72    }
73    .logo svg { flex-shrink: 0; }
74
75    .upload-btn {
76      display: inline-flex;
77      align-items: center;
78      gap: 6px;
79      padding: 7px 16px;
80      font-size: 13px;
81      font-weight: 500;
82      color: #fff;
83      background: var(--accent);
84      border: none;
85      border-radius: var(--radius-sm);
86      cursor: pointer;
87      transition: background .15s, box-shadow .15s;
```

```

88     white-space: nowrap;
89   }
90   .upload-btn:hover {
91     background: var(--accent-hover);
92     box-shadow: 0 2px 8px rgba(88,166,255,.3);
93   }
94   .upload-btn input { display: none; }
95
96   .topbar-spacer { flex: 1; }
97
98   .topbar-stats {
99     display: flex;
100    gap: 8px;
101    align-items: center;
102  }
103  .topbar-stat {
104    display: flex;
105    align-items: center;
106    gap: 5px;
107    font-size: 12px;
108    color: var(--text-secondary);
109    padding: 4px 10px;
110    background: var(--bg-secondary);
111    border-radius: 20px;
112    border: 1px solid var(--border);
113  }
114  .topbar-stat strong {
115    font-weight: 600;
116    color: var(--text);
117  }
118
119  /* — Upload screen ————— */
120  .upload-screen {
121    display: flex;
122    flex-direction: column;
123    align-items: center;
124    justify-content: center;
125    min-height: calc(100vh - 56px);
126    padding: 32px;
127  }
128
129  .upload-card {
130    width: 100%;
131    max-width: 520px;
132    background: var(--surface);
133    border: 1px solid var(--border);
134    border-radius: var(--radius);
135    box-shadow: var(--shadow-lg);
136    padding: 48px 40px;
137    text-align: center;
138  }
139
140  .upload-icon {
141    width: 64px;
142    height: 64px;
143    margin: 0 auto 24px;
144    border-radius: 16px;
145    background: var(--accent-bg);
146    border: 1px solid var(--accent-border);
147    display: flex;
148    align-items: center;
149    justify-content: center;
150  }
151
152  .upload-card h2 {
153    font-size: 18px;
154    font-weight: 600;
155    margin-bottom: 6px;
156    letter-spacing: -.01em;
157  }
158  .upload-card p {
159    font-size: 14px;
160    color: var(--text-secondary);
161    margin-bottom: 28px;
162  }
163
164  .drop-zone {
165    border: 2px dashed var(--border);
166    border-radius: var(--radius-sm);
167    padding: 36px 24px;
168    cursor: pointer;
169    transition: all .2s;
170    position: relative;
171  }
172  .drop-zone:hover,
173  .drop-zone.drag-over {
174    border-color: var(--accent);
175    background: var(--accent-bg);
176  }
177  .drop-zone.drag-over { transform: scale(1.01); }
178  .drop-zone p { margin: 0; }

```

```
179 .drop-zone .hint {
180   font-size: 12px;
181   color: var(--text-secondary);
182   margin-top: 8px;
183 }
184 .drop-zone input { display: none; }
185
186 .features {
187   display: grid;
188   grid-template-columns: 1fr 1fr;
189   gap: 12px;
190   margin-top: 28px;
191 }
192 .feature {
193   display: flex;
194   align-items: flex-start;
195   gap: 8px;
196   text-align: left;
197   font-size: 12px;
198   color: var(--text-secondary);
199 }
200 .feature svg { flex-shrink: 0; margin-top: 1px; }
201 .feature strong { color: var(--text); display: block; font-size: 13px; }
202
203 /* — Data view ————— */
204 .data-view { display: none; }
205 .data-view.active { display: block; }
206
207 /* Controls bar */
208 .ctrl-bar {
209   display: flex;
210   align-items: center;
211   gap: 12px;
212   padding: 14px 32px;
213   border-bottom: 1px solid var(--border);
214   background: var(--surface);
215   flex-wrap: wrap;
216 }
217
218 .tabs {
219   display: flex;
220   gap: 2px;
221   background: var(--bg);
222   border: 1px solid var(--border);
223   border-radius: var(--radius-sm);
224   padding: 3px;
225 }
226 .tab {
227   padding: 6px 14px;
228   border-radius: 6px;
229   font-size: 13px;
230   font-weight: 500;
231   cursor: pointer;
232   border: none;
233   background: transparent;
234   color: var(--text-secondary);
235   transition: all .15s;
236   white-space: nowrap;
237 }
238 .tab:hover { color: var(--text); }
239 .tab.active {
240   background: var(--surface);
241   color: var(--text);
242   box-shadow: var(--shadow-sm);
243 }
244
245 .search-wrap {
246   flex: 1;
247   min-width: 200px;
248   position: relative;
249 }
250 .search-wrap svg {
251   position: absolute;
252   left: 12px;
253   top: 50%;
254   transform: translateY(-50%);
255   color: var(--text-secondary);
256   pointer-events: none;
257 }
258 .search-wrap input {
259   width: 100%;
260   padding: 7px 14px 7px 36px;
261   font-size: 13px;
262   border: 1px solid var(--border);
263   border-radius: var(--radius-sm);
264   background: var(--bg);
265   color: var(--text);
266   outline: none;
267   transition: border-color .2s, box-shadow .2s;
268 }
269 .search-wrap input:focus {
```

```

270     border-color: var(--accent);
271     box-shadow: 0 0 0 3px rgba(88,166,255,.15);
272 }
273 .search-wrap input::placeholder { color: var(--text-muted); }
274
275 .badge {
276     display: inline-flex;
277     align-items: center;
278     padding: 5px 12px;
279     font-size: 12px;
280     font-weight: 500;
281     border-radius: 20px;
282     white-space: nowrap;
283 }
284 .badge-default {
285     background: var(--bg-secondary);
286     color: var(--text-secondary);
287     border: 1px solid var(--border);
288 }
289 .badge-match {
290     background: var(--accent-bg);
291     color: var(--accent);
292     border: 1px solid var(--accent-border);
293 }
294
295 /* — Table — */
296 .table-wrap {
297     overflow: auto;
298     max-height: calc(100vh - 130px);
299 }
300
301 table {
302     width: 100%;
303     border-collapse: collapse;
304     font-size: 13px;
305 }
306
307 thead { position: sticky; top: 0; z-index: 10; }
308
309 th {
310     background: var(--surface);
311     border-bottom: 1px solid var(--border);
312     padding: 10px 16px;
313     text-align: left;
314     font-weight: 600;
315     font-size: 11px;
316     color: var(--text-secondary);
317     text-transform: uppercase;
318     letter-spacing: .04em;
319     white-space: nowrap;
320     cursor: pointer;
321     user-select: none;
322     transition: color .15s;
323     position: relative;
324 }
325 th:first-child { padding-left: 24px; }
326 th:hover { color: var(--accent); }
327 th::after {
328     content: '';
329     position: absolute;
330     right: 0;
331     top: 25%;
332     height: 50%;
333     width: 1px;
334     background: var(--border);
335 }
336 th:last-child::after { display: none; }
337
338 .th-inner {
339     display: flex;
340     align-items: center;
341     gap: 4px;
342 }
343 .sort-arrow {
344     display: inline-flex;
345     font-size: 9px;
346     opacity: .4;
347     transition: opacity .15s;
348     color: var(--accent);
349 }
350 th.sorted .sort-arrow { opacity: 1; }
351
352 tbody tr {
353     border-bottom: 1px solid var(--border);
354     transition: background .1s;
355 }
356 tbody tr:nth-child(even) { background: var(--row-stripe); }
357 tbody tr:hover { background: var(--row-hover); }
358
359 td {
360     padding: 9px 16px;

```

```

361         white-space: nowrap;
362         max-width: 280px;
363         overflow: hidden;
364         text-overflow: ellipsis;
365     }
366     td:first-child {
367         padding-left: 24px;
368         font-weight: 500;
369     }
370     td.num {
371         text-align: right;
372         font-variant-numeric: tabular-nums;
373     }
374
375     mark {
376         background: rgba(210,153,34,.25);
377         color: var(--text);
378         border-radius: 2px;
379         padding: 0 2px;
380     }
381
382     /* empty */
383     .empty-state {
384         display: flex;
385         flex-direction: column;
386         align-items: center;
387         justify-content: center;
388         padding: 80px 20px;
389         color: var(--text-secondary);
390     }
391     .empty-state svg { margin-bottom: 16px; opacity: .4; }
392     .empty-state p { font-size: 15px; }
393
394     /* scrollbar */
395     ::-webkit-scrollbar { width: 6px; height: 6px; }
396     ::-webkit-scrollbar-track { background: transparent; }
397     ::-webkit-scrollbar-thumb { background: var(--border-light); border-radius: 3px; }
398     ::-webkit-scrollbar-thumb:hover { background: var(--text-muted); }
399
400     /* responsive */
401     @media (max-width: 640px) {
402         .topbar { padding: 0 16px; }
403         .ctrl-bar { padding: 10px 16px; }
404         .topbar-stats { display: none; }
405         .features { grid-template-columns: 1fr; }
406         .upload-card { padding: 32px 24px; }
407     }
408 </style>
409 </head>
410 <body>
411
412 <!-- Top bar -->
413 <div class="topbar">
414     <div class="logo">
415         <svg width="22" height="22" fill="none" viewBox="0 0 24 24">
416             <rect x="3" y="3" width="7" height="7" rx="1.5" fill="#58a6ff"/>
417             <rect x="14" y="3" width="7" height="7" rx="1.5" fill="#58a6ff" opacity=".4"/>
418             <rect x="3" y="14" width="7" height="7" rx="1.5" fill="#58a6ff" opacity=".4"/>
419             <rect x="14" y="14" width="7" height="7" rx="1.5" fill="#58a6ff" opacity=".2"/>
420         </svg>
421         Excel Parser
422     </div>
423
424     <label class="upload-btn" id="uploadBtn">
425         <svg width="14" height="14" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round" viewBox="0 0 24 24"><path d="M21 15v4a2 2 0 0 1 2 2h2a2 2 0 0 1 2 2v-4"/><polyline points="17 8 12 3 7 8"/></line x="12" y="3" x2="12" y2="15"/></svg>
426         Загрузить
427         <input type="file" id="fileInput" accept=".json">
428     </label>
429
430     <div class="topbar-spacer"></div>
431
432     <div class="topbar-stats" id="statsBar" style="display:none;">
433         <div class="topbar-stat">Файл: <strong id="statFile"></strong></div>
434         <div class="topbar-stat">Таблиц: <strong id="statTables"></strong></div>
435         <div class="topbar-stat">Строк: <strong id="statRows"></strong></div>
436     </div>
437 </div>
438
439 <!-- Upload screen -->
440 <div class="upload-screen" id="uploadScreen">
441     <div class="upload-card">
442         <div class="upload-icon">
443             <svg width="28" height="28" fill="none" viewBox="0 0 24 24">
444                 <path d="M14 2H6a2 2 0 0 2 2v16a2 2 0 0 2 2h12a2 2 0 0 2 2v-8Z" stroke="#58a6ff" stroke-width="1.5" stroke-linecap="round" stroke-linejoin="round"/>
445                 <polyline points="14 2 14 8 20 8" stroke="#58a6ff" stroke-width="1.5" stroke-linecap="round" stroke-linejoin="round"/>
446             </svg>
447         </div>

```

```

448     <h2>Загрузите JSON-файл</h2>
449     <p>Результат парсинга Excel для просмотра</p>
450
451     <div class="drop-zone" id="dropZone">
452         <svg width="32" height="32" fill="none" viewBox="0 0 24 24" style="display:block;margin:0 auto 12px;opacity:.4">
453             <path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4" stroke="#8b949e" stroke-width="1.5" stroke-linecap="round" stroke-linejoin="round"/>
454             <polyline points="17 8 12 3 7 8" stroke="#8b949e" stroke-width="1.5" stroke-linecap="round" stroke-linejoin="round"/>
455             <line x1="12" y1="3" x2="12" y2="15" stroke="#8b949e" stroke-width="1.5" stroke-linecap="round" stroke-linejoin="round"/>
456         </svg>
457         <p><strong style="color:var(--text)">Перетащите файл сюда</strong></p>
458         <p class="hint">или нажмите для выбора</p>
459         <input type="file" id="dropInput" accept=".json">
460     </div>
461
462     <div class="features">
463         <div class="feature">
464             <svg width="14" height="14" fill="none" viewBox="0 0 24 24"><path d="M20 6 9 17l-5-5" stroke="#3fb950" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"/></svg>
465             <div><strong>Поиск</strong>по всем столбцам</div>
466         </div>
467         <div class="feature">
468             <svg width="14" height="14" fill="none" viewBox="0 0 24 24"><path d="M20 6 9 17l-5-5" stroke="#3fb950" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"/></svg>
469             <div><strong>Сортировка</strong>клик по заголовку</div>
470         </div>
471         <div class="feature">
472             <svg width="14" height="14" fill="none" viewBox="0 0 24 24"><path d="M20 6 9 17l-5-5" stroke="#3fb950" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"/></svg>
473             <div><strong>Вкладки</strong>переключение таблиц</div>
474         </div>
475         <div class="feature">
476             <svg width="14" height="14" fill="none" viewBox="0 0 24 24"><path d="M20 6 9 17l-5-5" stroke="#3fb950" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"/></svg>
477             <div><strong>Без сервера</strong>всё в браузере</div>
478         </div>
479     </div>
480 </div>
481
482 <!-- Data view -->
483 <div class="data-view" id="dataView">
484     <div class="ctrl-bar">
485         <div class="tabs" id="tabBar"></div>
486         <div class="search-wrap">
487             <svg width="15" height="15" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" viewBox="0 0 24 24"><circle cx="11" cy="11" r="8"/><line x1="21" y1="21" x2="16.65" y2="16.65"/></svg>
488             <input type="text" id="searchInput" placeholder="Поиск по всем столбцам...">
489         </div>
490         <span class="badge badge-default" id="rowCount"></span>
491     </div>
492
493     <div class="table-wrap" id="tableWrap">
494         <table>
495             <thead id="thead"></thead>
496             <tbody id="tbody"></tbody>
497         </table>
498     </div>
499
500     <div class="empty-state" id="noResults" style="display:none;">
501         <svg width="40" height="40" fill="none" viewBox="0 0 24 24">
502             <circle cx="11" cy="11" r="8" stroke="#6e7681" stroke-width="1.5"/>
503             <line x1="21" y1="21" x2="16.65" y2="16.65" stroke="#6e7681" stroke-width="1.5" stroke-linecap="round"/>
504         </svg>
505         <p>Ничего не найдено</p>
506     </div>
507 </div>
508
509 <script>
510 /* — State — */
511 let data = null;
512 let curTable = 0;
513 let sort = { col: null, asc: true };
514 let query = '';
515
516 /* — DOM — */
517 const $ = id => document.getElementById(id);
518 const fileInput = $('fileInput');
519 const dropInput = $('dropInput');
520 const dropZone = $('dropZone');
521 const uploadBtn = $('uploadBtn');
522 const uploadScreen = $('uploadScreen');
523 const dataView = $('dataView');
524 const statsBar = $('statsBar');
525 const tabBar = $('tabBar');
526 const searchInput = $('searchInput');
527 const rowCount = $('rowCount');
528 const thead = $('thead');
529 const tbody = $('tbody');

```

```

531 const tableWrap = $('tableWrap');
532 const noResults = $('noResults');
533
534 /* — Upload ————— */
535 /* Fix: programmatically trigger the hidden input */
536 dropZone.addEventListener('click', () => dropInput.click());
537 uploadBtn.addEventListener('click', (e) => {
538   e.preventDefault();
539   fileInput.click();
540 });
541
542 fileInput.addEventListener('change', e => { if (e.target.files[0]) read(e.target.files[0]); });
543 dropInput.addEventListener('change', e => { if (e.target.files[0]) read(e.target.files[0]); });
544
545 dropZone.addEventListener('dragover', e => { e.preventDefault(); dropZone.classList.add('drag-over'); });
546 dropZone.addEventListener('dragleave', () => dropZone.classList.remove('drag-over'));
547 dropZone.addEventListener('drop', e => { e.preventDefault(); dropZone.classList.remove('drag-over'); if (e.dataTransfer.files[0]) read(e.dataTransfer.files[0]); });
548
549 /* Allow drop on entire page */
550 document.body.addEventListener('dragover', e => e.preventDefault());
551 document.body.addEventListener('drop', e => {
552   e.preventDefault();
553   if (e.dataTransfer.files[0] && e.dataTransfer.files[0].name.endsWith('.json')) read(e.dataTransfer.files[0]);
554 });
555
556 function read(file) {
557   const r = new FileReader();
558   r.onload = e => {
559     try { data = JSON.parse(e.target.result); boot(); }
560     catch (err) { alert('Bad JSON: ' + err.message); }
561   };
562   r.readAsText(file);
563 }
564
565 /* — Boot ————— */
566 function boot() {
567   uploadScreen.style.display = 'none';
568   dataView.classList.add('active');
569   statsBar.style.display = 'flex';
570
571   $('statFile').textContent = data.file || '-';
572   $('statTables').textContent = data.tables || 0;
573   $('statRows').textContent = (data.total_rows || 0).toLocaleString('ru-RU');
574
575   buildTabs();
576   switchTable(0);
577 }
578
579 /* — Tabs ————— */
580 function buildTabs() {
581   tabBar.innerHTML = '';
582   data.tables_data.forEach((t, i) => {
583     const b = document.createElement('button');
584     b.className = 'tab' + (i === 0 ? ' active' : '');
585     b.textContent = t.name || t.sheet || 'Таблица ${i+1}';
586     b.onclick = () => switchTable(i);
587     tabBar.appendChild(b);
588   });
589 }
590
591 function switchTable(i) {
592   curTable = i; sort = { col: null, asc: true }; query = ''; searchInput.value = '';
593   tabBar.querySelectorAll('.tab').forEach((t, j) => t.classList.toggle('active', j===i));
594   render();
595 }
596
597 /* — Render ————— */
598 function render() {
599   const tbl = data.tables_data[curTable];
600   const cols = tbl.columns;
601
602   thead.innerHTML = '<tr>' + cols.map((c,i) => {
603     const sorted = sort.col === i ? ' sorted' : '';
604     const arrow = sort.col === i ? (sort.asc ? '▲' : '▼') : '≡';
605     return '<th data-i=${i} class=${sorted}><span class=th-inner>${esc(c.name)}<span class=sort-arrow>${arrow}</span></th>';
606   }).join('') + '</tr>';
607
608   thead.querySelectorAll('th').forEach(th => {
609     th.onclick = () => {
610       const i = parseInt(th.dataset.i);
611       if (sort.col === i) sort.asc = !sort.asc;
612       else { sort.col = i; sort.asc = true; }
613       renderRows();
614     };
615   });
616
617   renderRows();
618 }
619

```

```

620 function renderRows() {
621   const tbl = data.tables_data[curTable];
622   const cols = tbl.columns;
623   let rows = [...tbl.rows];
624
625   if (query) {
626     const q = query.toLowerCase();
627     rows = rows.filter(r => cols.some(c => { const v = r[c.name]; return v != null && String(v).toLowerCase().includes(q
628   ); }));
629
630   if (sort.col !== null) {
631     const key = cols[sort.col].name;
632     const num = cols[sort.col].type === 'number';
633     rows.sort((a,b) => {
634       let va = a[key], vb = b[key];
635       if (va == null) return 1;
636       if (vb == null) return -1;
637       if (num && typeof va === 'number' && typeof vb === 'number') return sort.asc ? va - vb : vb - va;
638       va = String(va).toLowerCase(); vb = String(vb).toLowerCase();
639       if (va < vb) return sort.asc ? -1 : 1;
640       if (va > vb) return sort.asc ? 1 : -1;
641       return 0;
642     });
643   }
644
645   thead.querySelectorAll('th').forEach((th,i) => {
646     const arrow = th.querySelector('.sort-arrow');
647     if (i === sort.col) { th.classList.add('sorted'); arrow.textContent = sort.asc ? '▲' : '▼'; }
648     else { th.classList.remove('sorted'); arrow.textContent = '↕'; }
649   });
650
651   const total = tbl.rows.length;
652   if (query) {
653     rowCount.className = 'badge badge-match';
654     rowCount.textContent = `${rows.length} из ${total}`;
655   } else {
656     rowCount.className = 'badge badge-default';
657     rowCount.textContent = `${rows.length} строк`;
658   }
659
660   if (!rows.length) { tableWrap.style.display = 'none'; noResults.style.display = 'flex'; return; }
661   noResults.style.display = 'none'; tableWrap.style.display = 'block';
662
663   const MAX = 2000;
664   const show = rows.slice(0, MAX);
665   const numSet = new Set();
666   cols.forEach((c,i) => { if (c.type === 'number') numSet.add(i); });
667
668   tbody.innerHTML = show.map(r =>
669     '<tr>' + cols.map((c,i) => {
670       let v = r[c.name];
671       if (v == null) v = '';
672       const n = numSet.has(i) && typeof v === 'number';
673       const cls = n ? 'class="num"' : '';
674       const txt = n ? fmtNum(v) : esc(String(v));
675       return '<td${cls}>${hl(txt)}</td>';
676     }).join('') + '</tr>'
677   ).join('');
678
679   if (rows.length > MAX)
680     tbody.innerHTML += '<tr><td colspan="${cols.length}" style="text-align:center;padding:20px;color:var(--text-secondary
681 y)">... ещё ${rows.length - MAX} строк – уточните поиск</td></tr>';
682   }
683
684   /* — Search — */
685   let tid;
686   searchInput.addEventListener('input', e => {
687     clearTimeout(tid);
688     tid = setTimeout(() => { query = e.target.value.trim(); renderRows(); }, 180);
689   });
690
691   /* — Helpers — */
692   function esc(s) { return s.replace(/&/g,'&amp;').replace(/</g,'&lt;').replace(/>/g,'&gt;').replace(/"/g,'&quot;'); }
693   function hl(t) { if (!query) return t; return t.replace(new RegExp(query.replace(/[.*+?^${}()|\\[\]\\]/g,'\\$&'), 'gi'), '<mark
694   >${}&lt;/mark>'); }
695   function fmtNum(n) { return Number.isInteger(n) ? n.toLocaleString('ru-RU') : n.toLocaleString('ru-RU',{maximumFractionDigit
696   s:4}); }
697 </script>
698 </body>
699 </html>

```